Mohsin Iban Hossain

AIUB, Operating System Notes

Table of Contents

INTRODUCTION TO OPERATING SYSTEM

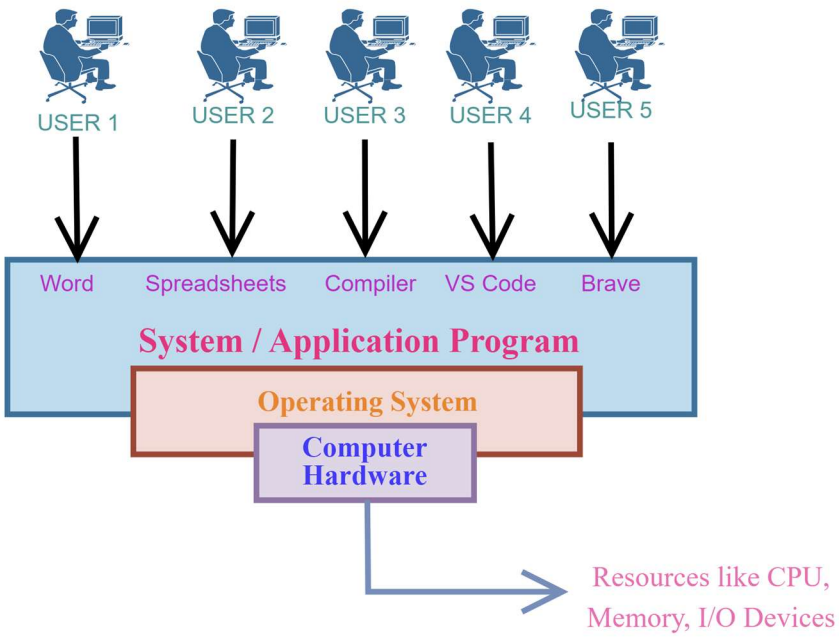
Computer System

⇨ A computer system has **four main components**:

1. **Hardware:**
   - Provides basic computing resources (CPU, memory, I/O devices).
2. **Operating System (OS):**
   - Controls and coordinates hardware usage among applications and users.
3. **Application Programs:**
   - Define how resources are used to solve user problems.
   - Examples: Word processors, compilers, web browsers, database systems, video games.
4. **Users:**
   - People, machines, or other computers interacting with the system.

Abstract View of Computer Components

- A computer can be viewed as layers:
  Hardware → Operating System → Application Programs → Users



Operating System (OS)

⇨ An **Operating System (OS)** is a **system software** that acts as an **interface** **between the user and the computer hardware**.

⇨ It manages hardware resources, controls program execution, and provides a convenient environment for users to run applications.

What Operating Systems Do

- Depends on **user perspective**:
  - Users want **convenience, ease of use, and good performance**.
  - They usually don't care about resource utilization.
- On **shared systems** (e.g., mainframes):
  - OS must efficiently manage resources for multiple users.
- **Role of OS:**
  - Acts as a **resource allocator** and **control program**.
  - Manages hardware and user program execution.

OS in Different Devices

❖ **Mobile Devices:** Resource-limited, optimized for usability & battery life.
  - Use touch, voice, and gesture interfaces.

❖ **Embedded Systems:** Run with little or no user interaction (e.g., in cars, appliances).

Defining Operating Systems

➢ "Operating System" covers **many roles** depending on system design and usage.

➢ Present in diverse devices — from **toasters to spacecraft**.

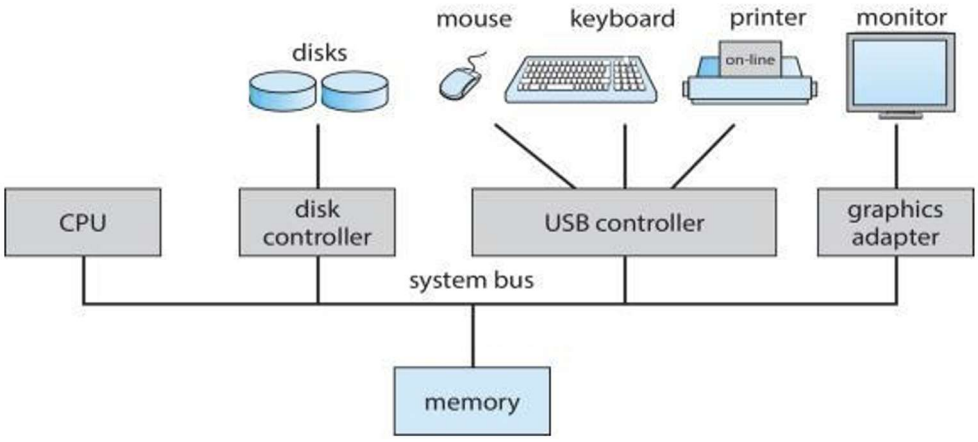
➢ Originally developed for **resource management** and **program control** in general-purpose computers.

Common Definitions

- No universal definition exists.
- "Everything a vendor ships when you order an OS" – broad definition.
- The **kernel** is the one program that always runs.
- Other parts include:
  - **System Programs:** Utilities shipped with the OS.
  - **Application Programs:** User-installed programs.
- **Middleware:** Software frameworks offering services like databases, multimedia, and graphics for app developers.

Computer System Organization

- A system has one or more **CPUs** and **device controllers** connected via a **common bus**.
- These share access to **main memory**.
- Enables **concurrent execution** of CPUs and I/O devices.



Computer-System Operation

- CPU and I/O devices execute **concurrently**.
- Each device **controller** manages a specific device type and uses a **local buffer**.
- The OS uses **device drivers** to manage communication.
- Data moves between **main memory** and **local buffers**.
- When I/O is complete, the **device controller sends an interrupt** to the CPU.

Some Important Terms

Bootstrap Program

- A small program stored in **ROM/firmware**.
- Runs automatically when the computer is powered on.
- **Initializes hardware** and **loads the operating system kernel** into main memory.
- Also called the **bootloader**.

Interrupts

- A **signal to the CPU** indicating that an event needs immediate attention.
- Temporarily **pauses the current task**, runs the **Interrupt Service Routine (ISR)**, then resumes.
- Helps handle **I/O operations, errors, or system requests** efficiently.
- Types: **Hardware**, **Software (Trap)**, **Vectored**, **Polling**

System Calls

- The way **user programs communicate with the operating system**.
- Allow programs to **request services** like file access, process control, or memory management.
- Switches the CPU from **user mode to kernel mode** for secure operation.
- Examples: `read()`, `write()`, `fork()`, `exec()`.

Common Functions of Interrupts

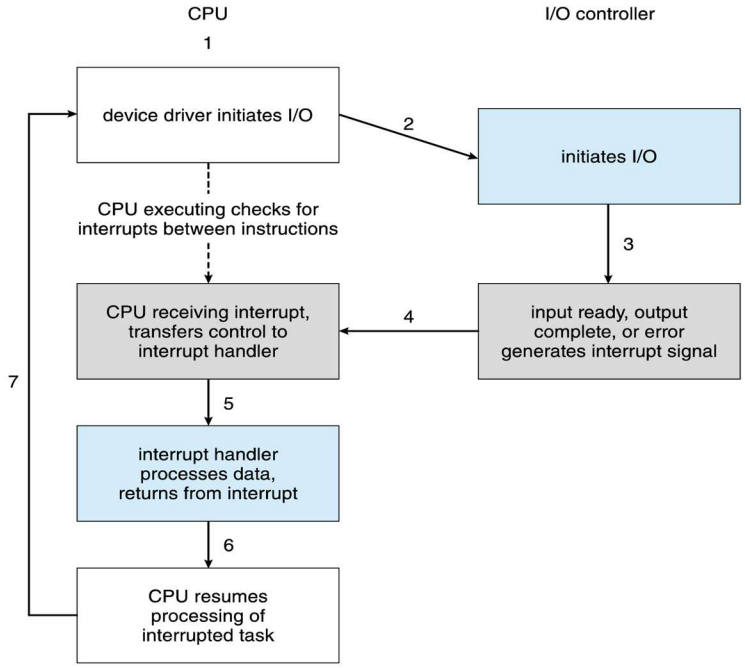
- Interrupts transfer control to the **Interrupt Service Routine (ISR)** via the **interrupt vector**.
- The CPU saves the **address of the interrupted instruction**.
- **Trap/Exception:** Software-generated interrupt (due to error or user request).
- OS is **interrupt-driven** — responds to hardware and software events.

Interrupt Handling

- OS **preserves CPU state** (registers and program counter).
- Determines interrupt type:
  - **Polling Interrupt:** CPU checks periodically if a device needs attention.
  - **Vectored Interrupt:** Device directly requests CPU attention.
- OS executes specific **service routines** for each interrupt type.

Interrupt-driven I/O Cycle

⇨ The CPU issues an I/O command → Device completes operation → Sends interrupt → CPU handles it → Execution resumes.



COMPUTER SYSTEM ARCHITECTURE

I/O Structure

- When input/output (I/O) starts, the program must wait until it's finished.
- The **CPU** may stay idle until an interrupt signals that I/O is complete.
- Only **one I/O operation** can happen at a time.
- A **system call** is used when a user program asks the OS to wait for I/O.
- The **Device-Status Table** keeps track of all I/O devices and their states.
- OS uses this table to check and update device status.

Storage Structure

⇨ **Main Memory:**

- Directly used by CPU.
- **Volatile** (data is lost when power is off).
- Made from DRAM.

⇨ **Secondary Storage:**

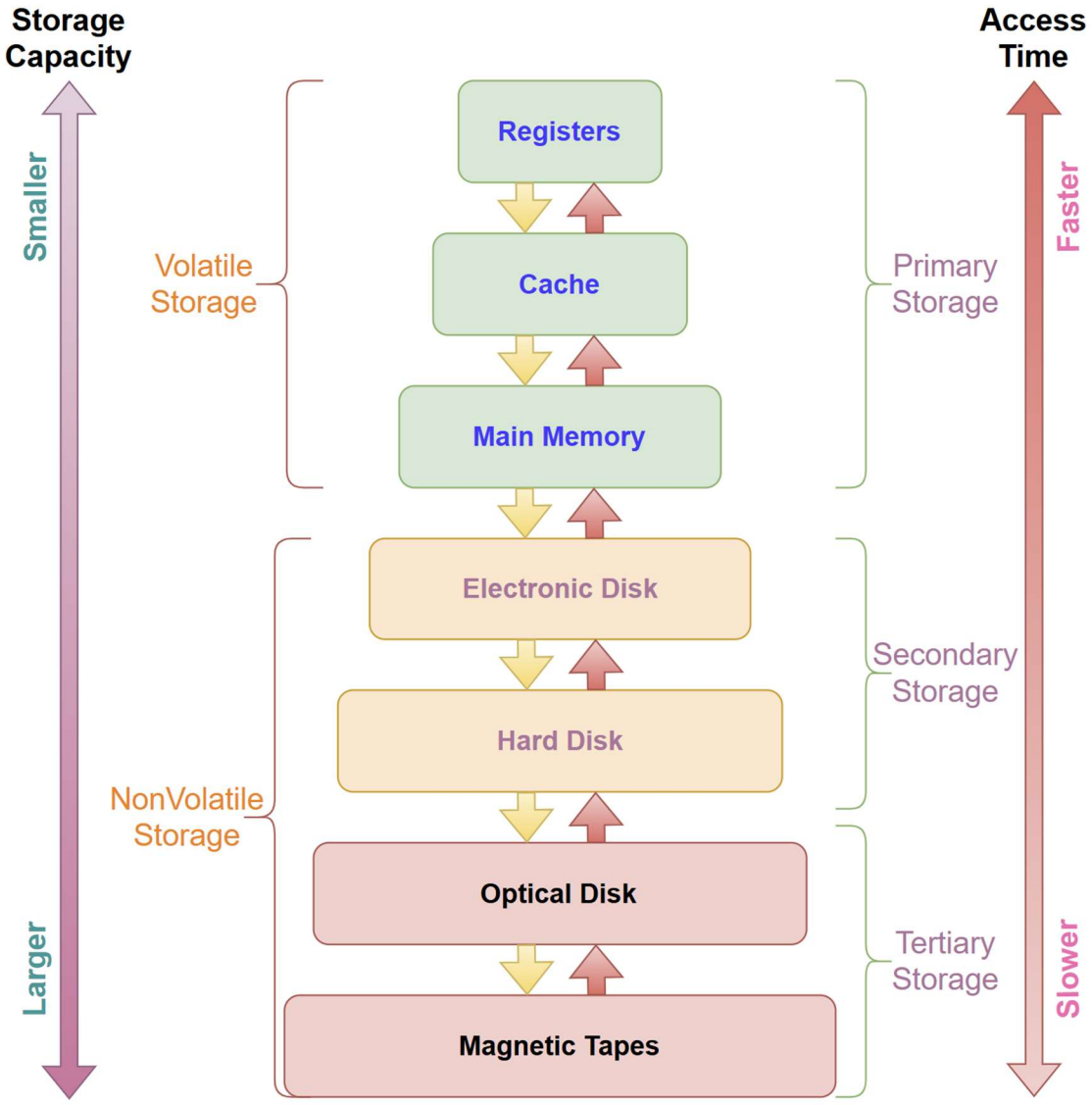
- Used to store data permanently.
- Example: **Hard Disk Drives (HDD)**
- Disk divided into **tracks and sectors**.

⇨ **Non-Volatile Memory (NVM):**

- Faster than hard disks.
- Keeps data even without power (e.g., SSDs).
- Becoming popular due to improved speed and lower cost.

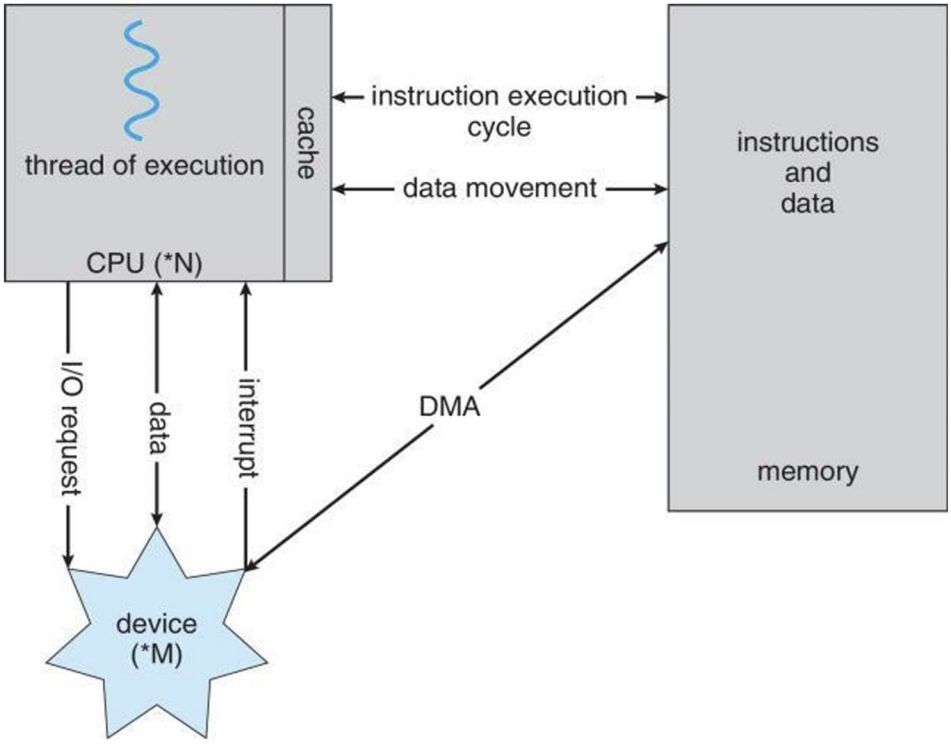
Storage Hierarchy

- Storage devices are organized by **speed, cost, and size**.
- **Caching:** Copying frequently used data to faster storage (e.g., RAM).
- **Device Drivers:** Help the OS communicate with hardware easily.



How a Modern Computer Works

- CPU, memory, and I/O units connected by system buses.
- Instructions and data share the same memory.



Direct Memory Access (DMA) Structure

- Used for **high-speed I/O devices**.
- Data transferred **directly between I/O device and memory** without CPU involvement.
- **Only one interrupt per data block**, not per byte.
- Reduces CPU overhead and increases efficiency.

Computer-System Architecture

- Most systems have one **general-purpose processor** and some **special-purpose processors**.

❖ **Types of computer system based on number of general-purpose processors:**

1. **Single Processor System**
2. **Multiprocessor System**
3. **Clustered System**

Single Processor System

- Contains **only one CPU (Central Processing Unit)**.
- All tasks—fetching, decoding, and executing instructions—are performed by this single processor.
- It can still have multiple cores, but only one main processor chip.
- **Examples:** Personal computers, laptops, small embedded systems.

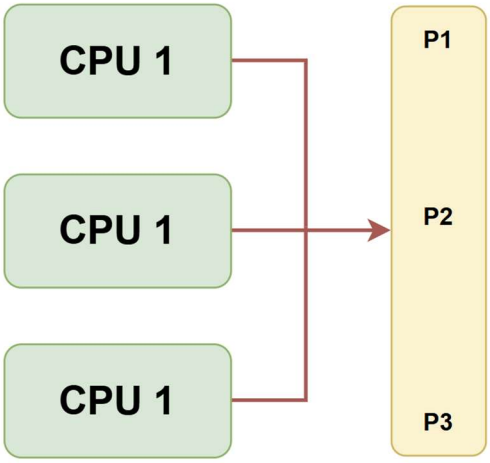
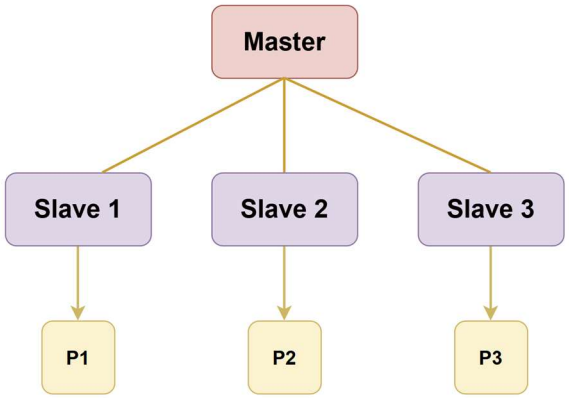
Multiprocessor System

- Contains **two or more processors** that share the same memory and work together.
- Processors can execute tasks **in parallel**, improving speed and reliability.
- Common in servers and high-performance computers.

  - **Advantages:**

    1. Increased throughput (more work done in less time)
    2. Economy of scale
    3. Higher reliability (if one processor fails, others can continue)

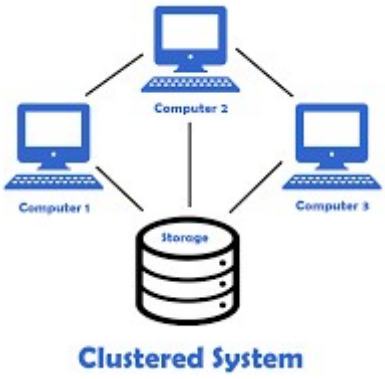
- **Types:**
  - **Asymmetric Multiprocessing (AMP):** Each processor assigned a specific task.
  - **Symmetric Multiprocessing (SMP):** All processors perform all tasks.

Symmetric Multiprocessing	Asymmetric Multiprocessing
	

Clustered System

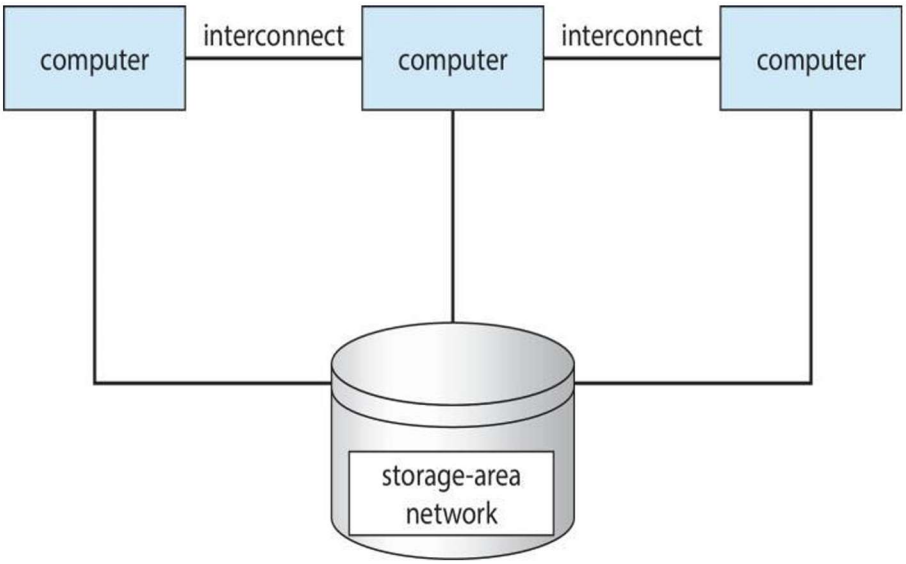
- Consists of **multiple independent computers (nodes)** working together as a single system.
- Each node has its **own processor, memory, and storage**, but they are connected via a **high-speed network**.
- Used for **high availability and load balancing**.

Examples: Cloud computing clusters, Google data centers.

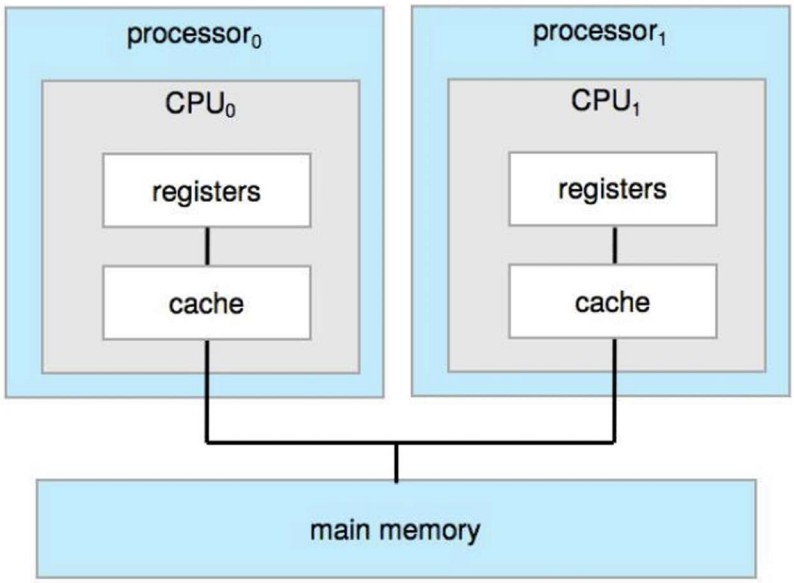


Types:

- **Asymmetric Clustering:** One node in standby mode.
- **Symmetric Clustering:** All nodes active and monitor each other.

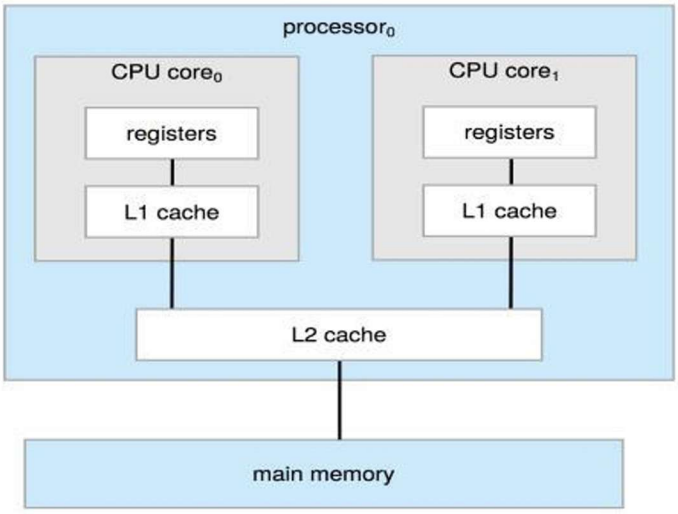


Symmetric Multiprocessing Architecture



A Dual-Core Design

- A single chip containing **multiple cores (CPUs)**.
- Enables **parallel processing** within one physical processor.
- Increases performance and power efficiency.



Operating-System Operations

- **Bootstrap Program:** Initializes system and loads the kernel.
- **Kernel:** Core of the OS; manages system operations.
- **System Daemons:** Background services started after kernel loads.
- **Interrupts:**
  - Hardware interrupt – from devices.
  - Software interrupt (trap/exception) – from programs (e.g., division by zero).
- **System Call:** Request from a program to OS for a specific service.

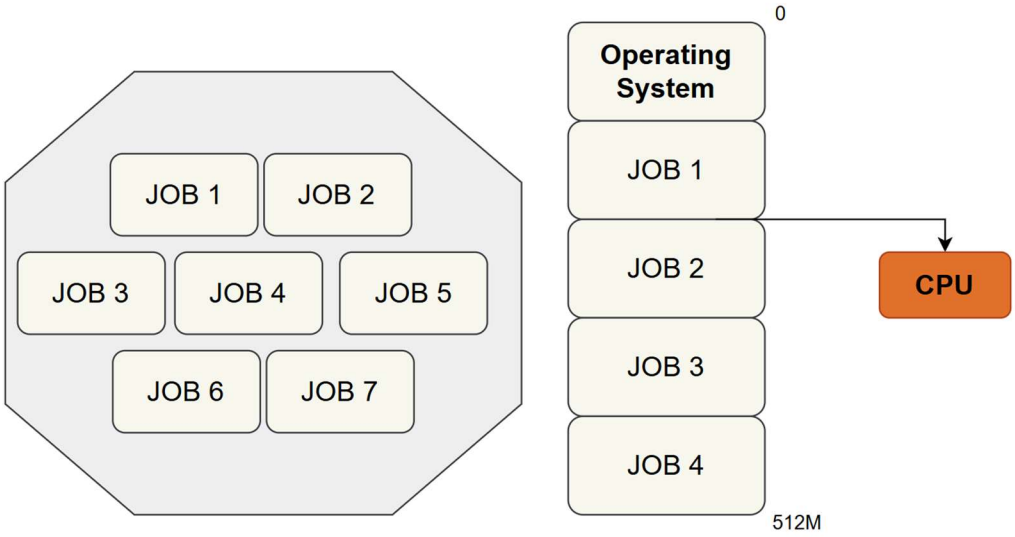
Multiprogramming and Multitasking

- **Multiprogramming:**
  - Keeps the CPU busy by running multiple programs at once.
  - When one program waits (for I/O), another can run.

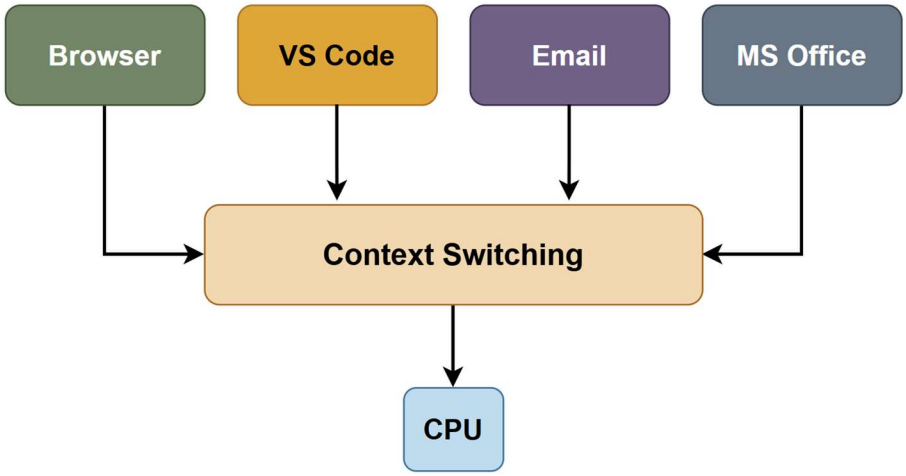
❖ Multitasking (Time-Sharing):

- CPU switches between tasks very quickly.
- Makes it seem like programs run at the same time.
- Response time should be **less than 1 second**.
- Each user has at least one active **process** in memory.
- **CPU Scheduling:** Chooses which job runs next.
- **Virtual Memory:** Allows execution of processes not fully in memory.

Multiprogramming



Multitasking

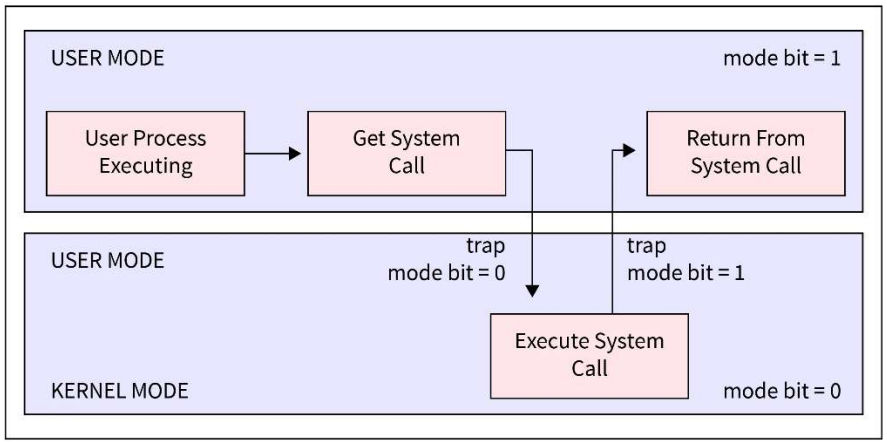


Dual-Mode and Multimode Operation

- ⇨ Protects the system from user errors.
- ⇨ Two main modes:
  - **User Mode:** Runs normal applications.
  - **Kernel Mode:** Runs OS code and privileged instructions.
- ⇨ The **mode bit** tells which mode the CPU is in.
- ⇨ Some CPUs have **extra modes** for virtual machines (VMM mode).

Transition from User to Kernel Mode

- **Timer interrupt** prevents any process from hogging the CPU.
- OS sets a **counter** that decreases with each clock tick.
- When counter reaches zero → **interrupt generated** → OS regains control.



Process Management

- A **process** is a program that's currently running.
- Needs CPU, memory, I/O, and files.
- **Single-threaded:** Runs one instruction at a time.
- **Multi-threaded:** Can run multiple parts at the same time.
- **OS manages processes by:**
    1. Creating and deleting processes.
    2. Suspending and resuming them.
    3. Handling communication between processes.
    4. Preventing deadlocks.

Memory Management

- OS decides which programs and data stay in memory.
- **Main Goals:**
    - Keep CPU busy.
    - Give fast response to users.
- **Tasks:**
    - Track which memory areas are used.
    - Move programs/data in or out of memory.
    - Allocate and free memory when needed.

File-System Management

- The OS organizes data in the form of **files and folders**.
- Hides hardware details and gives a simple view of storage.
- **Main tasks:**
    - Create/delete files and folders.
    - Control who can access what.
    - Store files safely on secondary storage.
    - Backup important data.

Caching

- Keeps **frequently used data** in faster memory (cache).
- When data is needed:
    - If it's in the cache → use it directly (fast).
    - If not → get it from slower storage and copy to cache.
- OS manages what stays in the cache and what gets replaced.

OPERATING-SYSTEM STRUCTURES

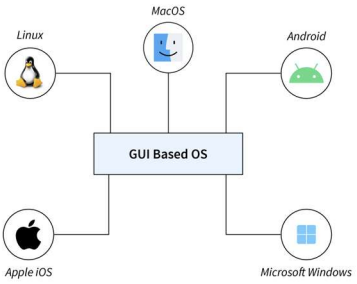
Operating System Services

⇨ Operating systems provide services that make using the computer easier and help programs run properly.

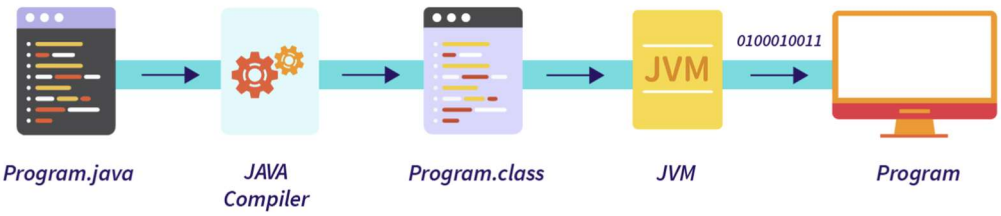
- **These services are divided into two groups:**

A. Services for Users and Programs

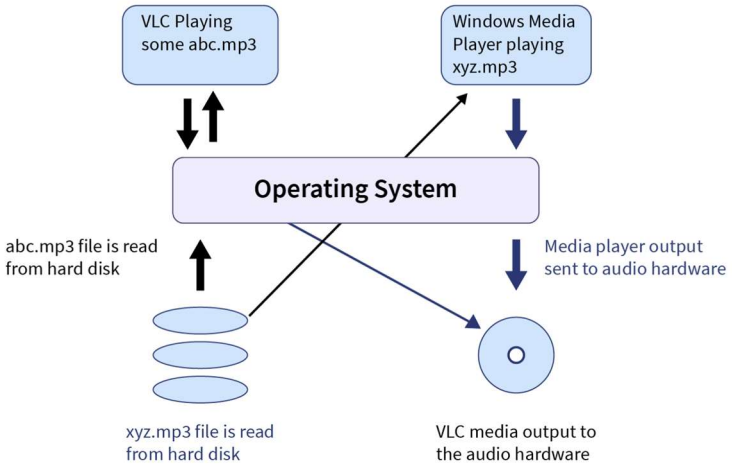
1. **User Interface (UI):**
   - Allows users to interact with the computer.
   - Can be **CLI (Command Line)**, **GUI (Graphical Interface)**, or **Touchscreen**.

Graphical Interface	GUI Based OS
	

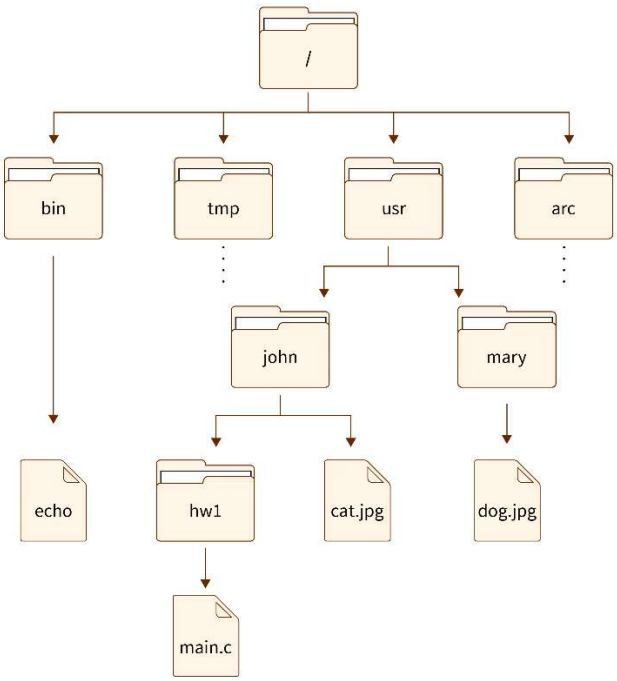
2. **Program Execution:**
   - The OS loads and runs programs.
   - It also handles ending a program (normally or with errors).

How Java Programming Language Execute?


3. **I/O Operations:**
    - OS manages input/output (like reading files or sending data to a printer).

Demonstrated of how input and output works.


4. **File-System Management:**
    - Helps in creating, deleting, reading, and writing files.
    - Manages folders and file permissions.

Structure provided below for better visualization.


5. **Communication:**
    - Programs or processes can exchange information.
    - Can happen between processes on the same system or over a network.
    - Uses **message passing** or **shared memory**.

6. **Error Detection:**
    - OS continuously checks for errors in CPU, memory, I/O, or user programs.
    - Helps maintain a stable and consistent system.

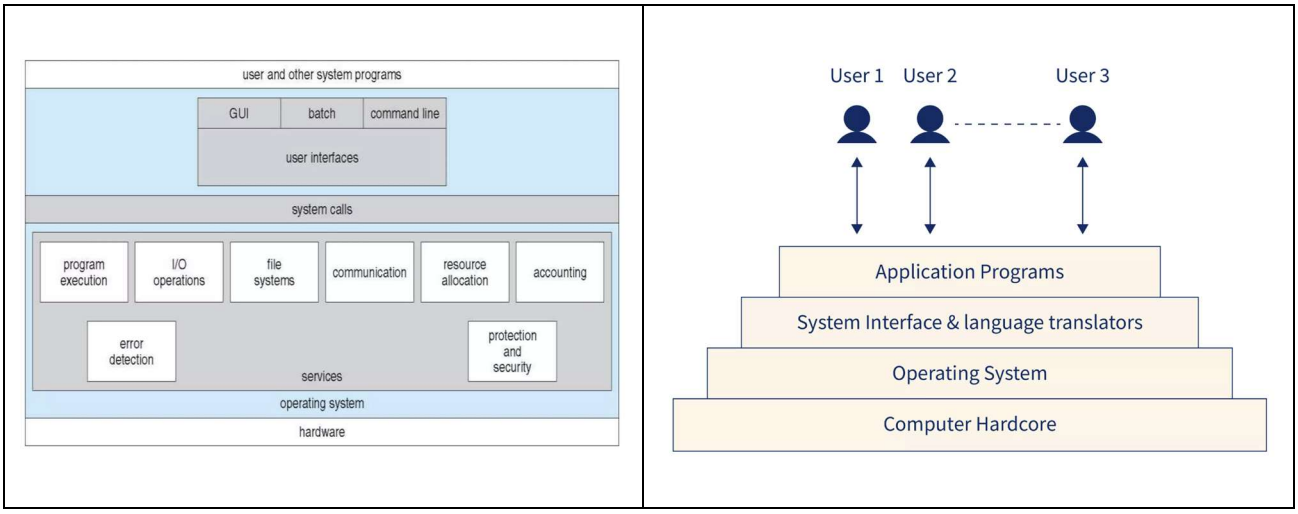
B. Services for System Efficiency

1. **Resource Allocation:**
    - OS divides resources (CPU, memory, I/O devices) among users or processes.

2. **Logging:**
    - Keeps records of resource usage by users and programs.

3. **Protection and Security:**
    - Ensures that processes do not interfere with one another.
    - Protects data and verifies users through **authentication**.

A View of Operating System Services



User Interfaces

❖ Command Line Interface (CLI):

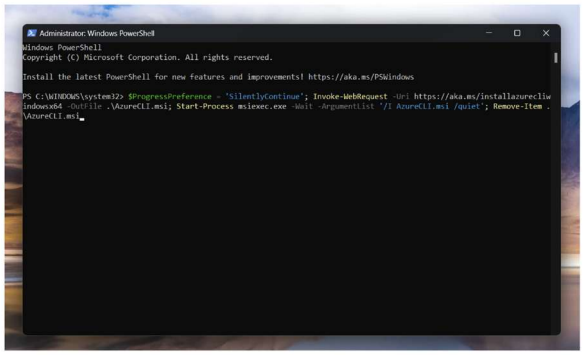
- Text-based interface for typing commands.
- Common in UNIX/Linux (e.g., Bash, Shell).
- Can run built-in commands or external programs.

❖ Graphical User Interface (GUI):

- Uses icons, windows, and menus.
- Example: Windows (Command + GUI), macOS (Aqua GUI), Linux (KDE, GNOME).

❖ Touchscreen Interface:

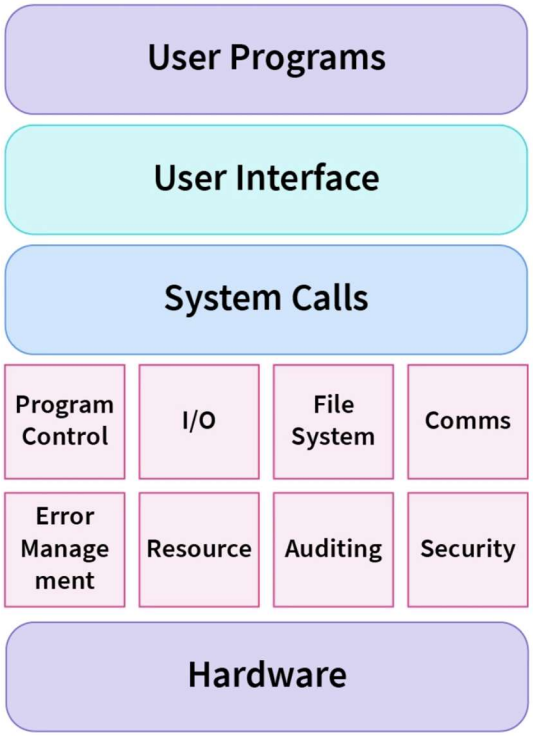
- Used in phones and tablets.
- Uses gestures (tap, swipe) and virtual keyboards.
- May also include voice commands.

CLI:	GUI:
	
Touchscreen Interface:	
	

System Calls

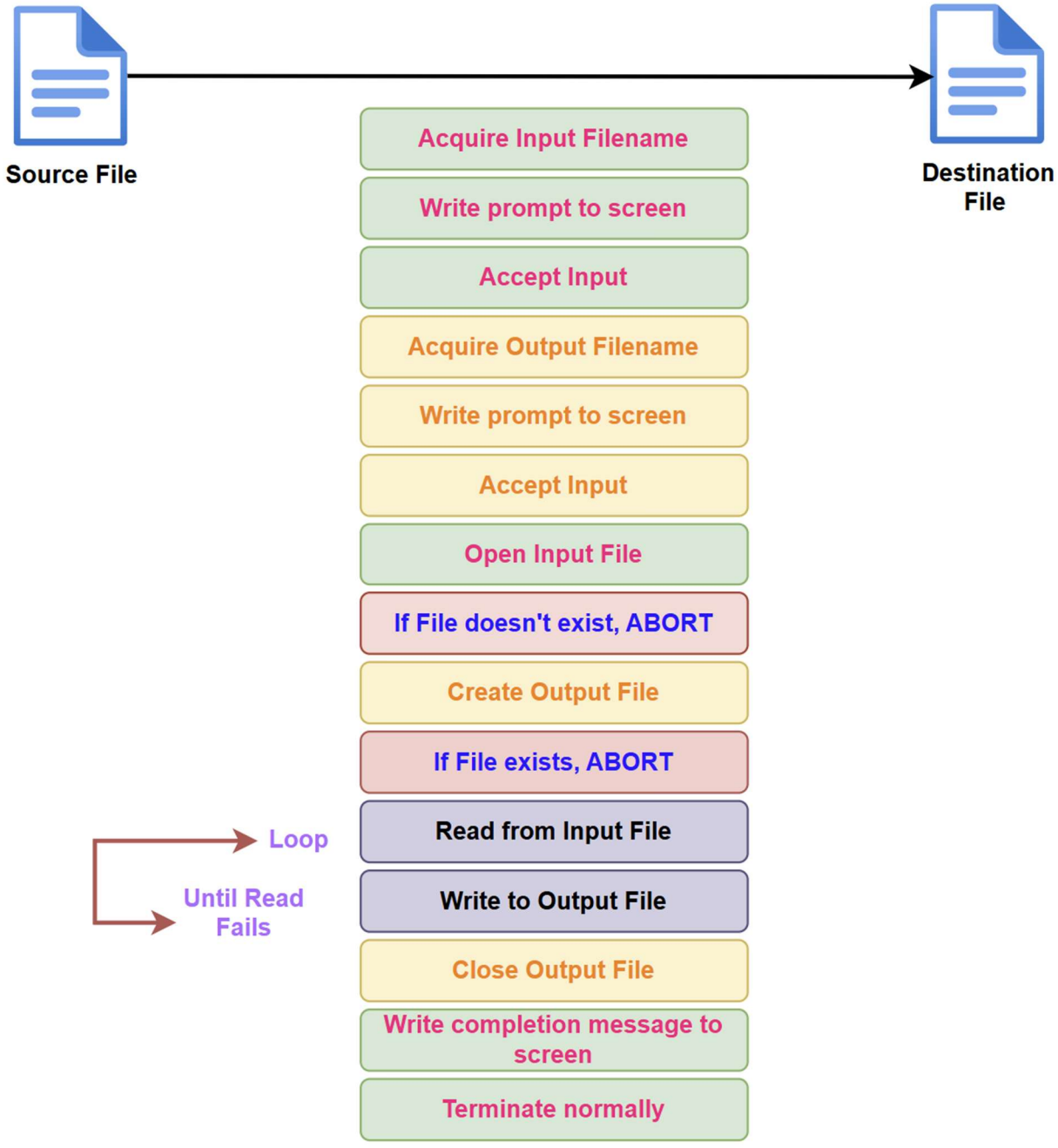
⇨ System calls are how a program asks the operating system to do something.

- They are **the link between a program and the OS**.
- Usually written in **C or C++**.
- Used through **APIs (Application Programming Interfaces)** like:
  - **Win32 API** (Windows)
  - **POSIX API** (UNIX/Linux/macOS)
  - **Java API** (Java Virtual Machine)



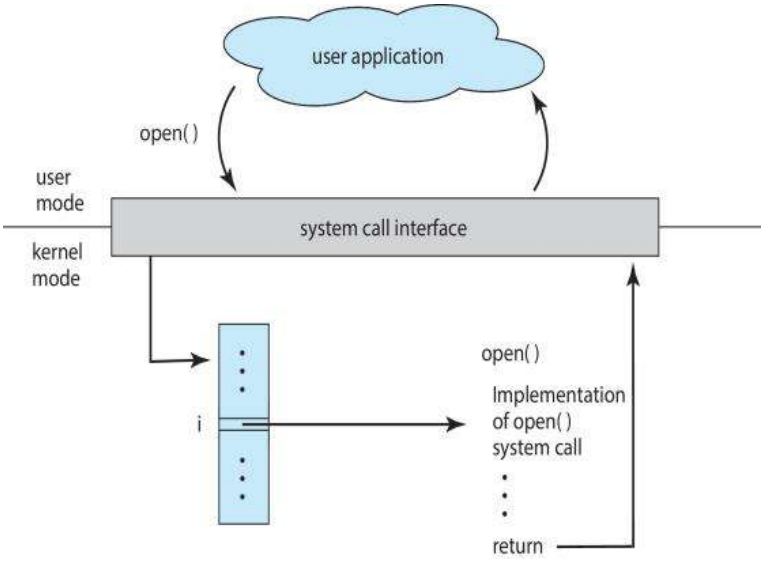
Example of a System Call

- **System call sequence to copy the contents of one file to another file**



System Call Implementation

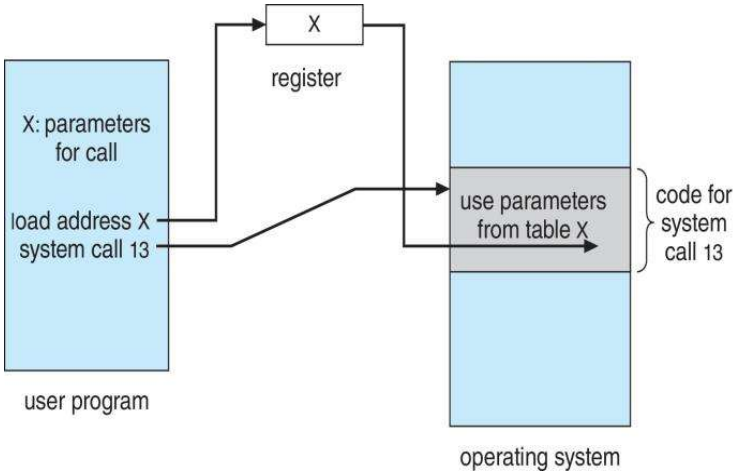
- Each system call has a unique **number**.
- The OS uses a **system call table** to find the correct function.
- The program doesn't need to know how the call works — it just uses the API.



Passing Parameters to the OS

⇨ Programs send data (parameters) to the OS in three ways:

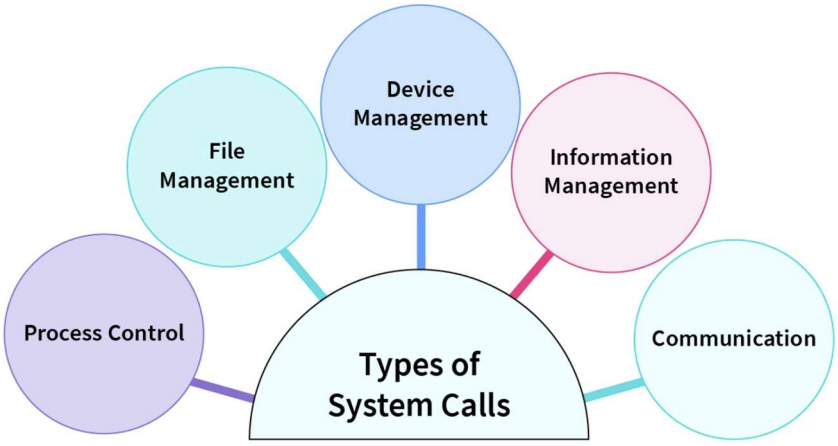
1. **Registers** – Fastest but limited.
2. **Memory Block/Table** – Address of a block with data is passed. (Used in Linux/Solaris.)
3. **Stack** – Parameters pushed onto stack by program, taken off by OS.



Types of System Calls

⇨ There are mainly 5 types of system calls available:

1. Process Control
2. File Management
3. Device Management
4. Information Maintenance
5. Communication



A. Process Control

- Create or end processes (`fork`, `exit`)
- Load and execute programs
- Get or Set attributes
- Wait, signal, or synchronize processes
- Allocate or free memory

B. File Management

- Create, delete, open, read, write, close files
- Get or Set attributes

C. Device Management

- Request or release devices
- Read/write from devices
- Logically Attach or detach hardware
- Get or Set attributes

D. Information Maintenance

- Get/set system time or data
- Access system information (like process status)

E. Communication

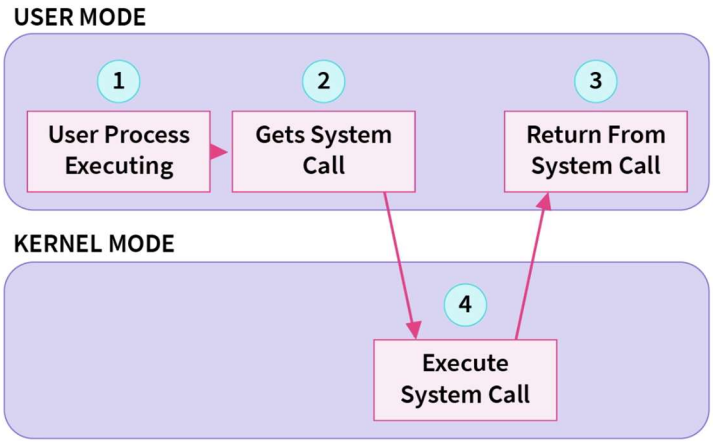
- Create or remove communication links
- Send and receive messages
- Manage shared memory

F. Protection

- Set or get access permissions
- Control which users can access certain data

How Does the System Call Work?

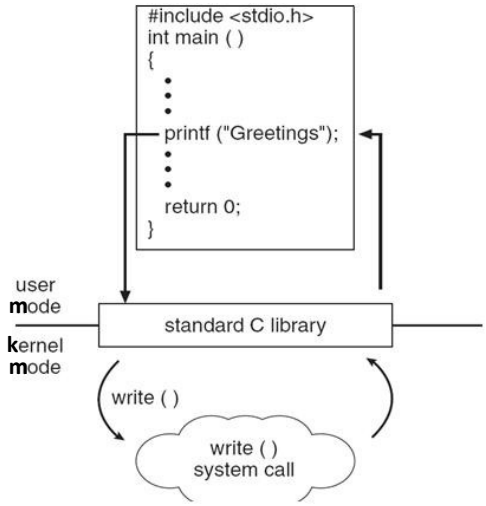
- A program normally runs in **User Mode**.
- OS runs in **Kernel Mode**.
- When a program needs OS services (like file access, memory, I/O), it makes a **system call**.
- The system call switches the CPU from **User Mode → Kernel Mode**.
- OS performs the required task **in kernel mode**.
- After completing, control switches back **Kernel Mode → User Mode**.
- The program continues running in **user mode**.



Example of System Calls in Windows and Unix

Types of System Calls	Windows	Linux
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Management	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Management	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()

Standard C Library Example

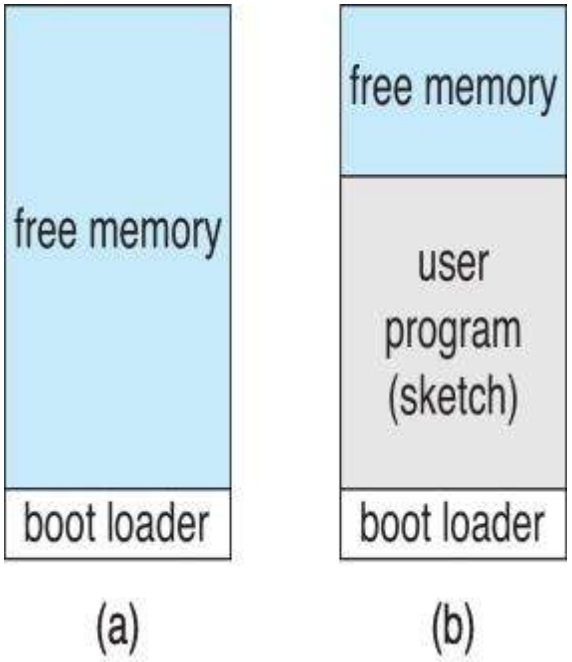


How C Library Interacts with System Calls

- The **C standard library (libc)** provides functions that act as a **bridge** to system calls in UNIX/Linux.
- When a program calls **printf()**, it does **not directly** call the OS.
- Instead, the **C library** catches the printf() call.
- The library internally uses the **write() system call** to send output to the screen.
- The value returned by **write()** is then given back to **printf()**, and finally returned to the user program.

Arduino Example

- Arduino is **single-tasking** → runs only one program at a time.
- It has **no operating system**.
- Programs (called **sketches**) are uploaded through **USB** into its **flash memory**.
- Arduino has **one memory space** shared by everything.
- A **bootloader** starts first and then loads the user program.
- When the program ends, the **bootloader/shell runs again**.



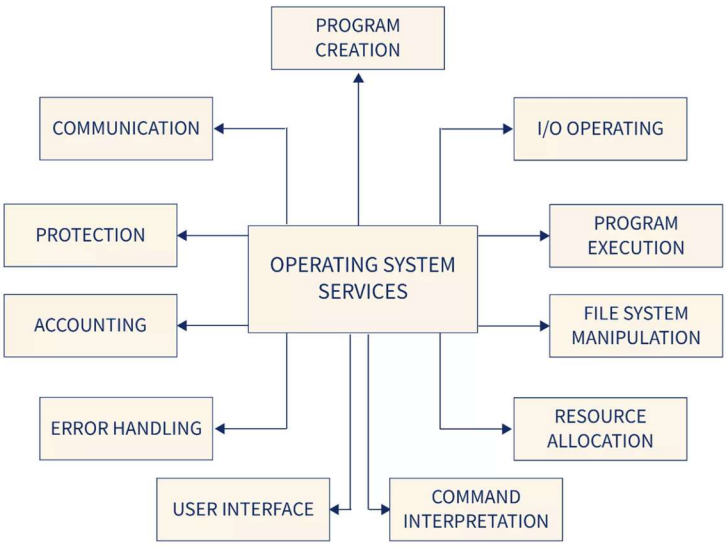
At system startup Running a program

System Services (System Programs)

⇨ These are programs provided by the OS to make life easier for users and programmers.

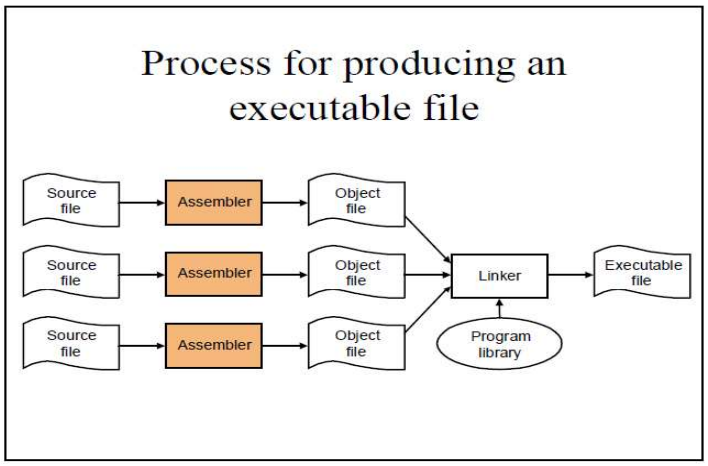
❖ Types of System Services:

1. **File Management:** Create, copy, delete, rename, or print files.
2. **Status Information:** Check system info (date, time, memory, performance).
3. **File Modification:** Use text editors to edit or search files.
4. **Programming Support:** Compilers, assemblers, debuggers, interpreters.
5. **Program Loading and Execution:** Load and run programs.
6. **Communication Tools:** Send messages, browse the web, transfer files.
7. **Background Services:**
   - Start when system boots.
   - Handle things like printing, error logs, or scheduling.
   - Run as **daemons** or **services**.
8. **Application Programs:**
   - Normal user apps (e.g., browsers, games).
   - Not part of the OS itself.



Linker

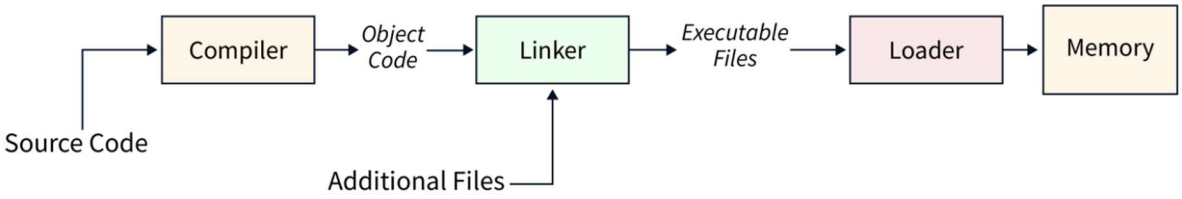
- A **linker** combines object modules into a single executable program.
- Used in **modular programming** to simplify large programs.
- After compilation, it **joins all object files**.
- Also called **link editor** or **binder**.



Loader

- A **loader** copies programs from storage to **main memory**.
- Makes programs **ready for execution**.
- It is a **special type of system program**.

The Role of the Linker and Loader



Why Apps Are OS Specific

❖ Application Compatibility & System Calls

- Apps compiled on one OS usually **cannot run on another OS**.
- Because each OS has its **own system calls**, **own file formats**, and **own execution environment**.
- **Cross-Platform Apps:**
    - Use **interpreted languages** (Python, Ruby) → interpreter exists on multiple OS.
    - Use languages with a **Virtual Machine (VM)** (Java) → same bytecode runs everywhere.
    - Use **standard compiled languages** (C/C++) → need to be **compiled separately for each OS**.

❖ ABI (Application Binary Interface)

- ABI = low-level interface between binary programs and the OS.
- It defines how binary code interacts with:
    - CPU architecture
    - System calls
    - Data types, calling conventions, registers, etc.
- ABI is like the **binary-level version of an API**, ensuring compiled programs can run on a specific OS + CPU architecture.

OS Design and Implementation

❖ Design Goals

- **User goals:** Easy, fast, reliable, and safe.
- **System goals:** Easy to build, maintain, efficient, and flexible.

❖ Policy vs Mechanism

- **Policy:** What to do.
- **Mechanism:** How to do it.
- Separating them makes the OS more flexible and easier to modify.

❖ Implementation

- Early operating systems were written in **assembly language**.
- Later moved to **system programming languages** like Algol and PL/1.
- Modern OSes mostly use **C and C++**.
- Real OS code is usually a **mix of languages**:
    - **Assembly** → lowest-level parts (boot code, context switching).
    - **C** → majority of the kernel.
    - **C/C++** → system utilities and components.
    - **Scripting languages** (Perl, Python, Shell) → tools, scripts, system management.

➢ Portability vs Performance

- **High-level languages** = easier to move (port) to other hardware.
- But they are generally **slower** than low-level languages.

Operating System Structures

⇨ OS structure refers to the **basic design/model** used to build an operating system. Different OS structures offer different ways to organize OS components. Common OS structures include:

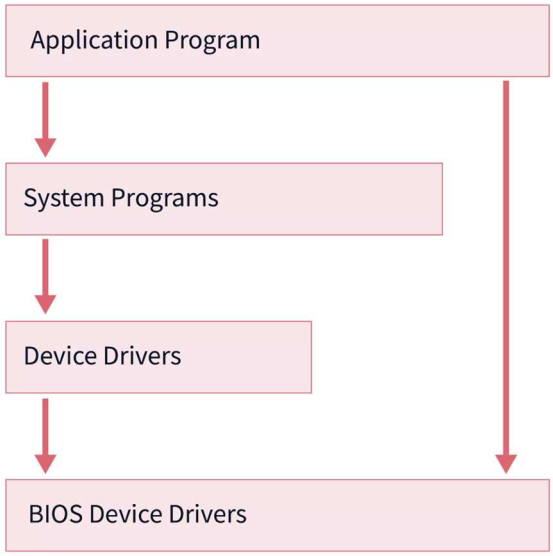
1. **Simple Structure**
2. **Monolithic Structure**
3. **Layered Structure**
4. **Microkernel Structure**

Simple Structure

- Simplest OS structure; used only for **small, limited systems**.
- Not well-defined and has **poor separation** between OS functions.
- Programs can directly access **I/O routines**, causing **security risks**.

❖ **Example: MS-DOS, early** UNIX**.**

❖ **MS-DOS has simple layers:**



❖ **Advantages of Simple Structure:**

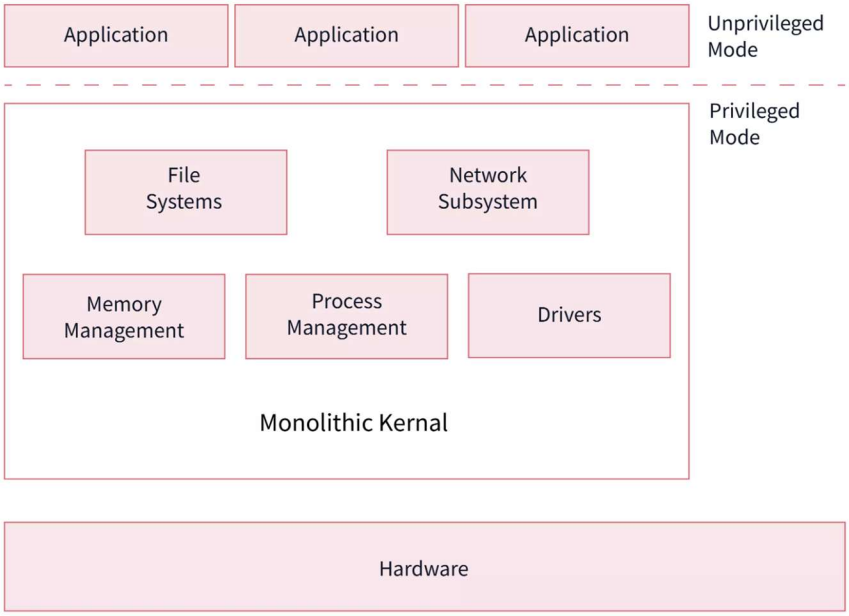
- Easy to **design and implement**
- Good for **small and limited systems**
- Layers can **interact easily**

❖ **Disadvantages of Simple Structure:**

- Low abstraction → user can access system routines directly (**security risk**)
- If one program crashes → **entire OS crashes**
- Not suitable for large or complex systems

Monolithic Structure

- A **monolithic OS** has one large kernel that manages **everything**:
  - File management
  - Memory management
  - Device management
  - Process management
- The **kernel is the core** of the OS and provides all essential services.
- In this structure, the kernel has **direct access** to all hardware (keyboard, mouse, CPU, etc.).
- Also called a **monolithic kernel**.



❖ Advantages of monolithic Structure:

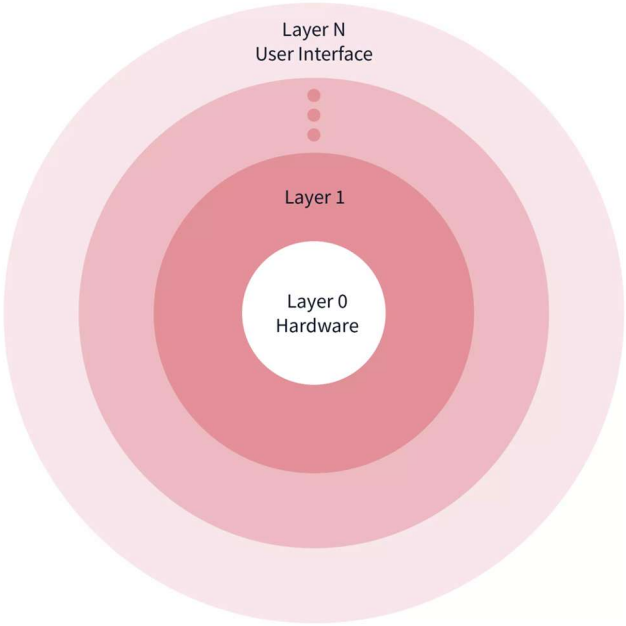
- **Fast execution** → kernel has direct access to hardware
- Efficient for **batch processing and time-sharing**
- All services in **one place** → no context switching overhead

❖ Disadvantages of monolithic Structure:

- Large kernel → **hard to maintain and debug**
- Any kernel bug can **crash the entire system**
- Poor modularity → changes affect many parts

Layered Structure

- The OS is divided into **layers**, from **Layer 0 (hardware)** to **Layer N (user interface)**.
- Higher layers use the services of the layers **below** them.
- Each layer has its **own functions** and provides **abstraction**.



❖ Advantages:

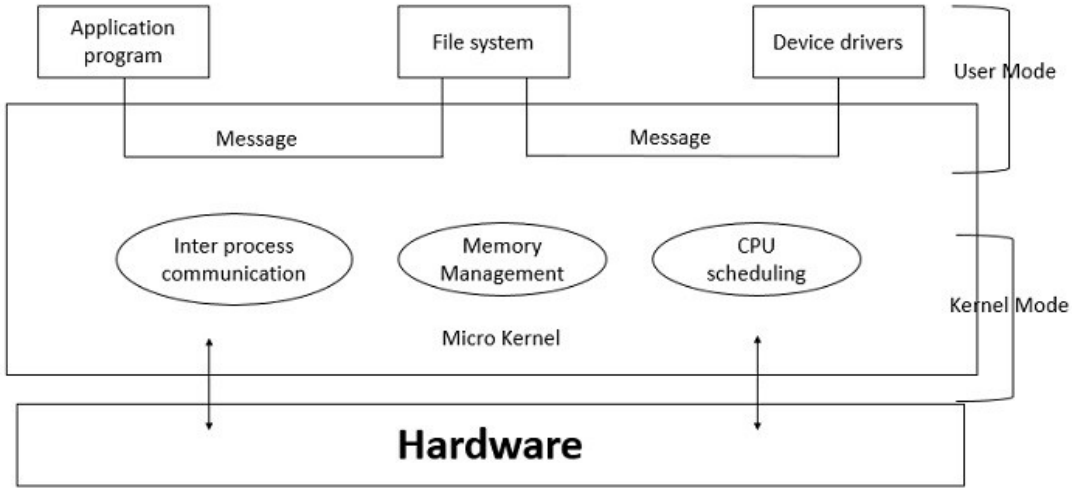
- **Easy to debug** → lower layers are tested first, then upper layers.
- **Clear structure** → each layer handles specific tasks.
- **Easy to maintain** → change in one layer doesn't affect others.
- **Better organization** and separation of functionality.

❖ Disadvantages:

- Can be **less efficient** due to layer-to-layer communication
- Designing proper layers is **difficult**
- Some layers may need to bypass hierarchy for performance

Micro-kernel Structure

- The **microkernel** keeps only **essential kernel components** in the kernel.
- Non-essential services are moved to **user-space programs** (still part of the OS).
- Each microkernel component is **independent and isolated**.
- **Communication** between modules happens via **message passing**.



❖ Advantages:

- **More secure** → isolated components
- **More reliable** → failure in one component doesn't crash OS
- Easier to **maintain and extend**
- Supports **modularity** and **flexibility**

❖ Disadvantages:

- **Slower** than monolithic OS due to frequent user-kernel communication
- More **complex design** needed to manage inter-process communication
- Requires careful coordination between components

Modular Systems

❖ Loadable Kernel Modules (LKMs)

- Modern OSes use **LKMs** to make the kernel **modular**.
- Uses an **object-oriented approach**: each core component is **separate**.
- Components **communicate over defined interfaces**.
- Modules are **loadable on demand** within the kernel.
- Similar to **layered structure**, but **more flexible**.
- **Examples:** Linux, Solaris

Hybrid Systems

- Combine features of multiple structures for performance and flexibility.
- Examples:
  - **Windows:** Mostly monolithic + microkernel parts.
  - **macOS:** Hybrid (Mach + UNIX + modular parts).
  - **Linux:** Monolithic but modular.

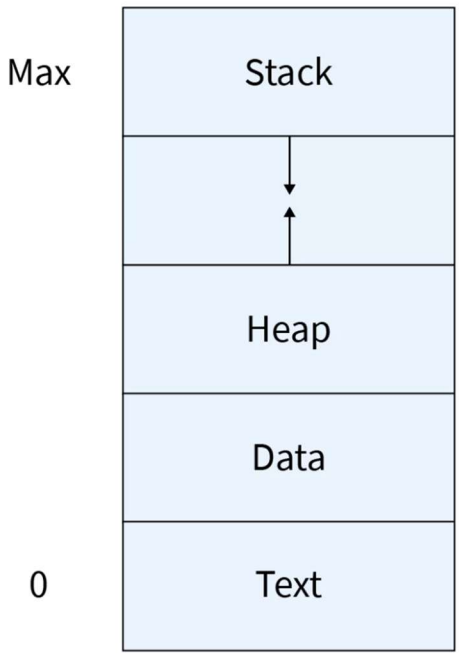

PROCESS

Process

- A **process** is an **instance of a program in execution**.
- It is the **basic unit of work** that the OS can **schedule and execute**.
- **Program** → passive (stored on disk).
  Process → active (loaded into memory + running).
- One program can have many processes (e.g., multiple users running Chrome).

Components of a Process

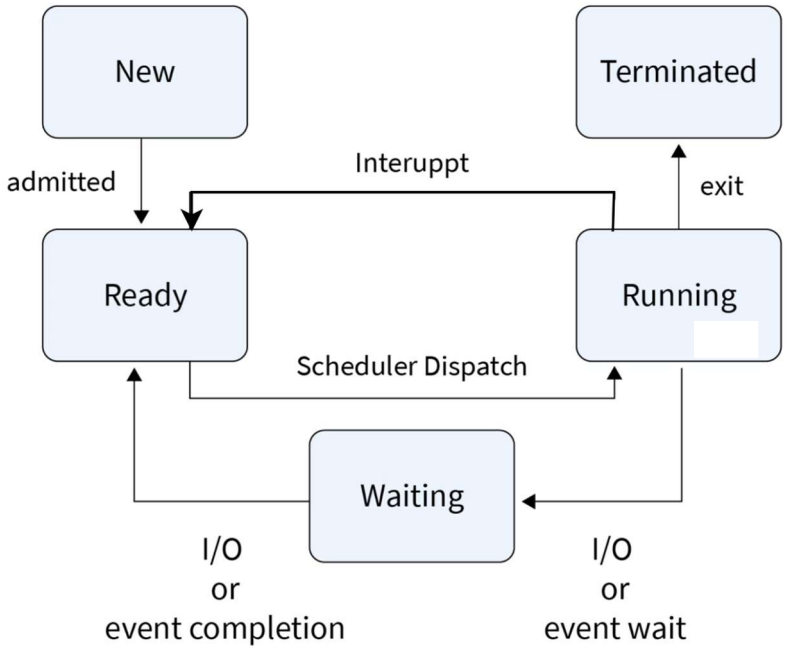
1. **Program Code / Text** → Instructions to be executed.
2. **Data** → Variables and information used by the process.
3. **Stack** → Stores **temporary data** like function parameters and return addresses.
4. **Heap** → Stores **dynamically allocated memory**.
5. **Process Control Block (PCB)** → Contains process info: state, priority, memory usage, etc.



Process State

- A process goes through different states during its lifetime:

1. **New**
   - Process is just created.
   - Not yet assigned CPU or resources.
2. **Ready**
   - Process is prepared to run.
   - Waiting in the ready queue for CPU.
3. **Running**
   - Process is **currently executing** on the CPU.
4. **Waiting (Blocked)**
   - Process cannot continue until an event happens (e.g., I/O completes).
5. **Terminated**
   - Process has **finished execution** and is removed from the OS.

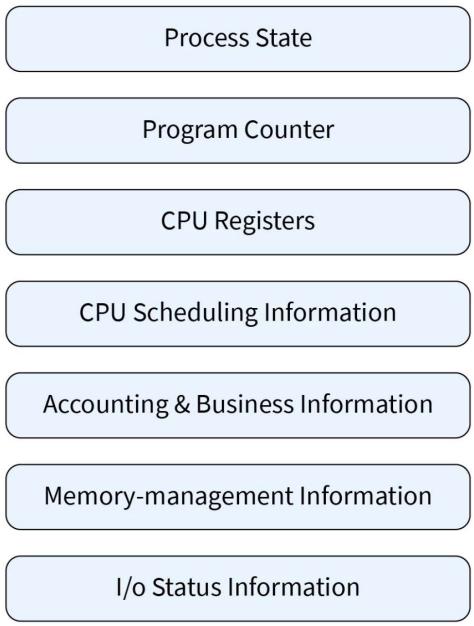


Process Control Block (PCB)

- ⇨ The OS uses **PCB** to manage each process.
- ⇨ Every process has **its own PCB**.
- ⇨ PCB stores all important information needed to **create, schedule, and control** a process.

❖ Components of PCB

1. **Process ID (PID)**
   - Unique identifier for locating the process.
2. **Process State**
   - Current state: new, ready, running, waiting, or terminated.
3. **Program Counter**
   - Stores the address of the **next instruction** to execute.
4. **CPU Registers**
   - Saves all register values when the process is not running (accumulators, index registers, instruction register, condition codes, etc.)
5. **CPU Scheduling Information**
   - Priority of the process
   - Scheduling queue pointers
   - Other scheduling parameters
6. **Accounting Information**
   - CPU time used
   - Number of jobs
   - Resource usage details
7. **Memory Management Information**
   - Base and limit register values
   - Page tables / segment tables
8. **I/O Status Information**
   - List of I/O devices assigned
   - Status of I/O operations



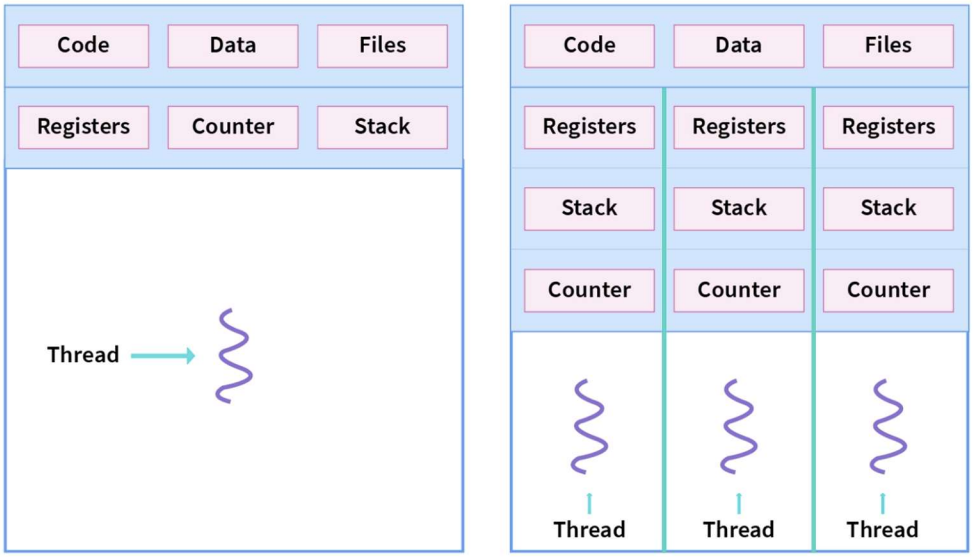
Threads

⇨ A **thread** is a small part of a process that can run tasks one after another. It helps a program do **multiple things at the same time** and improves performance.

❖ **Each thread has its own:**

- **Program counter** – remembers the next instruction to run
- **Stack** – stores temporary data
- **Registers** – for quick calculations

But all threads inside the same process **share the same code, data, and files**, which is why threads are often called **lightweight processes**.



Single-threaded process Multithreaded process

❖ **Example:** While **playing a movie** on a device the **audio** and **video** are controlled by **different threads** in the background.

Process Scheduling

⇨ Process scheduling is how the operating system decides **which process will use the CPU next**.

❖ Why Process Scheduling?

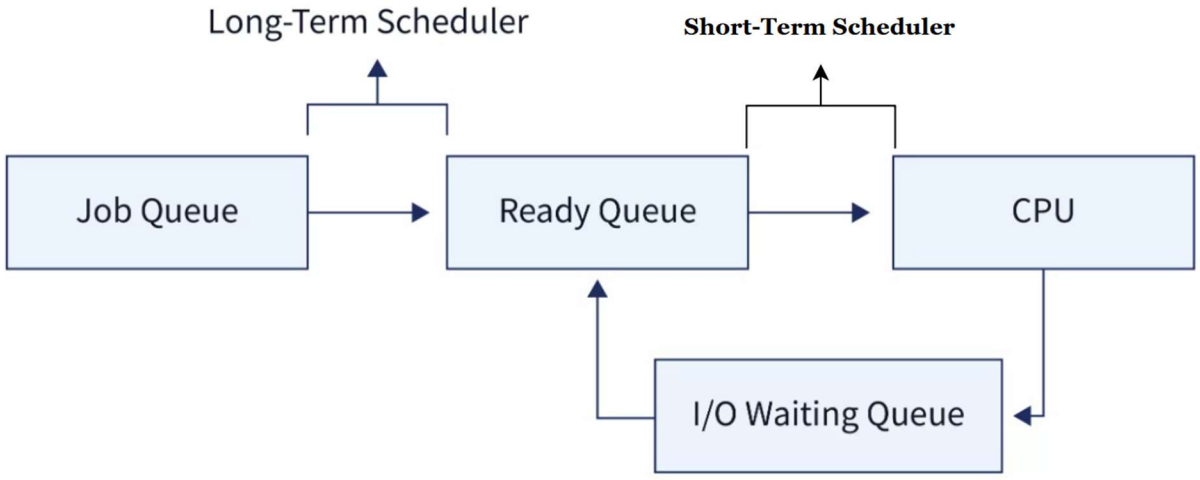
- To **maximize CPU use**
- To **give every process a fair chance**
- To **switch quickly** between processes

❖ How it Works

- Processes wait in the **Ready Queue**.
- The **Scheduler** selects one process from this queue.
- The selected process runs on the **CPU**.
- If it performs I/O or its time slice ends, it goes back to the Ready Queue.
- The scheduler again picks another process.

❖ Types of Process Schedulers

1. **Long-term scheduler** → Chooses which jobs enter the ready queue.
2. **Short-term scheduler** → Chooses which ready process runs on the CPU.
3. **Medium-term scheduler** → Suspends and resumes processes (swapping).



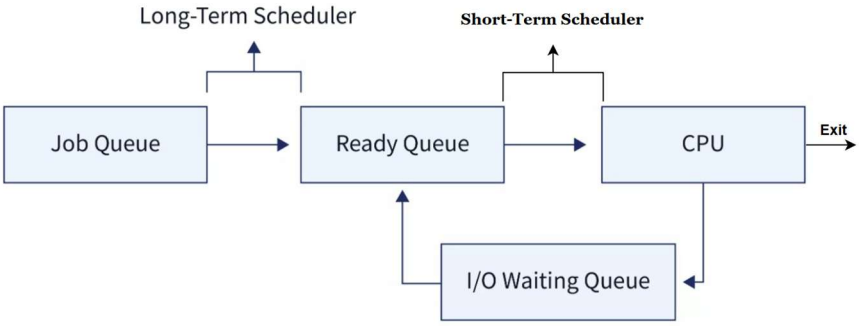
Scheduling Queues

⇨ OS Maintains **scheduling queues** of processes

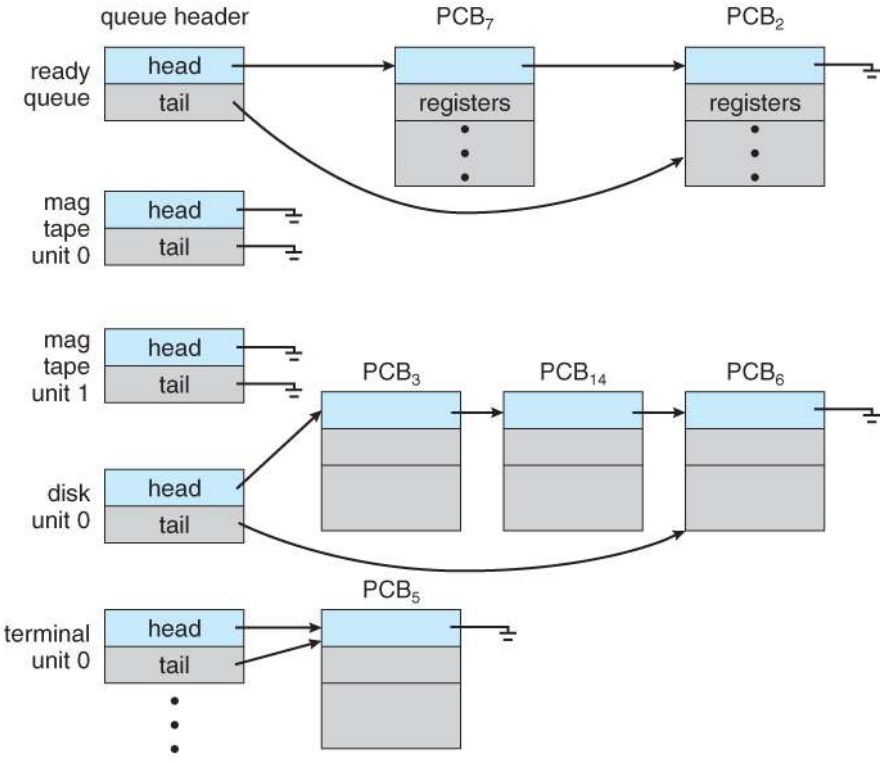
- **Job Queue:** All processes in the system (waiting to enter main memory).
- **Ready Queue:** Processes that are in main memory and ready to run on the CPU.
- **Device Queue:** Processes waiting for an I/O device (like printer, disk, etc.).

❖ Process Movement

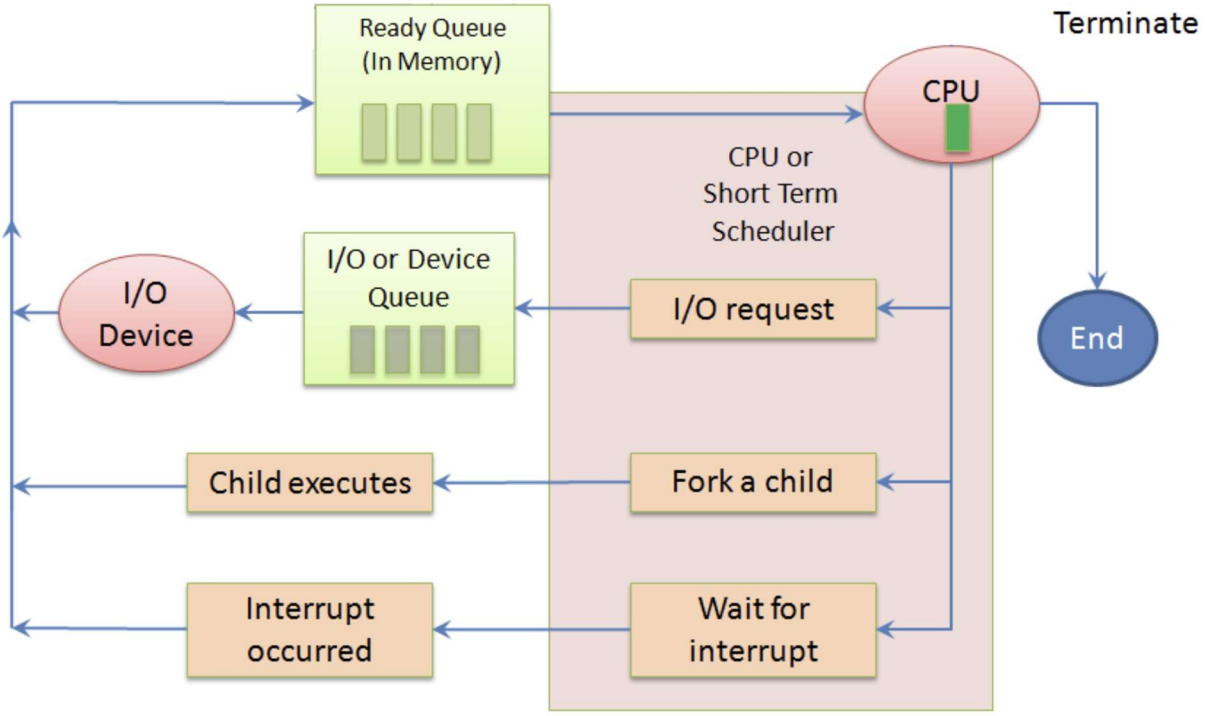
➢ Processes move from one queue to another depending on what they need:
Job Queue → Ready Queue → Running → Device Queue (if I/O needed) → Back to Ready Queue → Terminated.



Ready Queue and Various I/O Device Queues



Queueing-diagram representation of process scheduling



Context Switch

- Context switching is the process where the OS **pauses one running process**, saves its state, and **loads another process** to run on the CPU. This helps the system run many processes **as if they are working at the same time**.

❖ Why Do We Need Context Switching?

- **Multitasking:** Allows multiple processes to share CPU time smoothly.
- **Efficient CPU Use:** If one process is waiting for I/O, the CPU runs another process instead of sitting idle.
- **Fairness:** Ensures all processes get a fair chance on the CPU.
- **System Calls & Interrupts:** When the system needs to switch from user mode to kernel mode.

When Does Context Switching Happen? (Triggers)

1. **Time Quantum Over:** In Round Robin, when a process's time slice ends.
2. **Higher Priority Process Arrives:** OS preempts the lower priority process.
3. **I/O Request:** A process waiting for I/O gives up the CPU.
4. **Interrupts Occur:** Timer interrupts, keyboard interrupts, network interrupts, etc.
5. **Process Finishes:** When a process ends, CPU switches to the next one.

Role of PCB in Context Switching

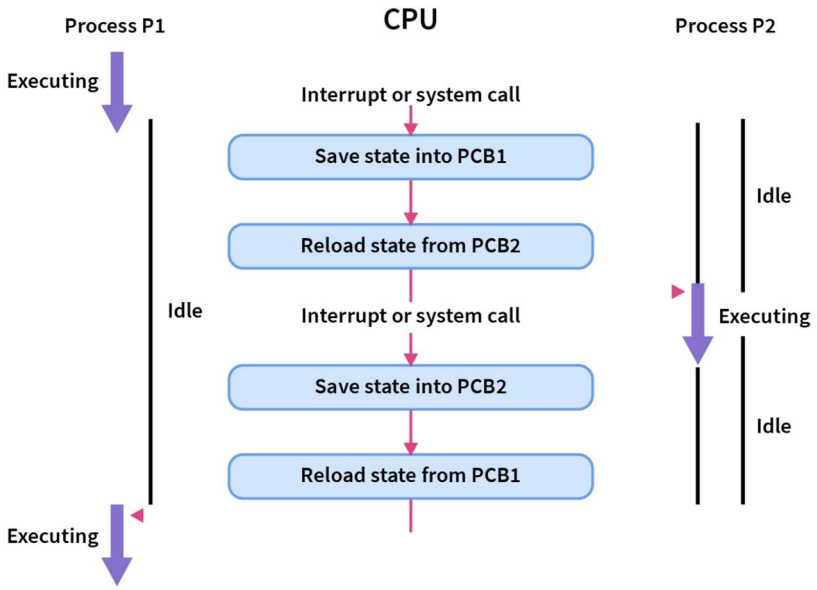
⇨ PCB stores all important information about the process:

- Registers
- Program counter
- CPU scheduling info
- Memory info
- I/O status

❖ **During context switch:**

1. OS **saves** the current process's info in its PCB.
2. OS **loads** the next process's PCB values.
3. CPU continues the new process from where it stopped earlier.

State Diagram of Context Switching



Short-Term vs Long-Term Scheduler

❖ Short-Term Scheduler (CPU Scheduler)

- Decides **which process runs next** on the CPU.
- Runs **very frequently** (every few milliseconds).
- Must be **fast**.
- Sometimes it is the **only scheduler** in the system.

❖ Long-Term Scheduler (Job Scheduler)

- Decides **which processes enter the Ready Queue** from secondary memory.
- Runs **rarely** (every few seconds or minutes).
- Can be **slow**.
- Controls the **degree of multiprogramming** (how many processes are in memory).

Types of Processes

❖ I/O-bound process:

- Spends most time doing **I/O**
- Has **many short CPU bursts**

❖ CPU-bound process:

- Spends most time doing **computations**
- Has **few long CPU bursts**

Mobile Operating Systems – Process Handling

❖ Older Mobile OS (e.g., early iOS versions)

- Allowed **only one process to run** at a time.
- All other apps were **suspended**.

❖ iOS (Starting from iOS 4):

1. Foreground Process

- The app currently **visible on screen**.
- Fully active and controlled by the user.

2. Background Processes

- Apps stay **in memory**, but not on the display.
- They run with **strict limits**, such as:
  - **Single short task** allowed
  - **Receive notifications**
  - **Specific long-running tasks**, e.g.,
    - Audio playback
    - GPS/navigation
    - VoIP

❖ Android:

- Supports **foreground and background apps** with **fewer restrictions**.
- Background processes use a **Service** to continue work.

➢ Service in Android

- Runs **without a user interface**.
- Can keep running **even if the app is suspended**.
- Uses **very little memory**.

Operations on Processes

⇨ These are the main tasks the OS performs to manage processes:

1. **Process State Management**
2. **Process Scheduling**
3. **Context Switching**
4. **Process Creation**
5. **Process Termination**
6. **Inter-Process Communication (IPC)**
7. **Process Execution**
8. **Process Synchronization**
9. **Process Priority Management**
10. **Process Accounting & Monitoring**

Process Creation

1. What is Process Creation?

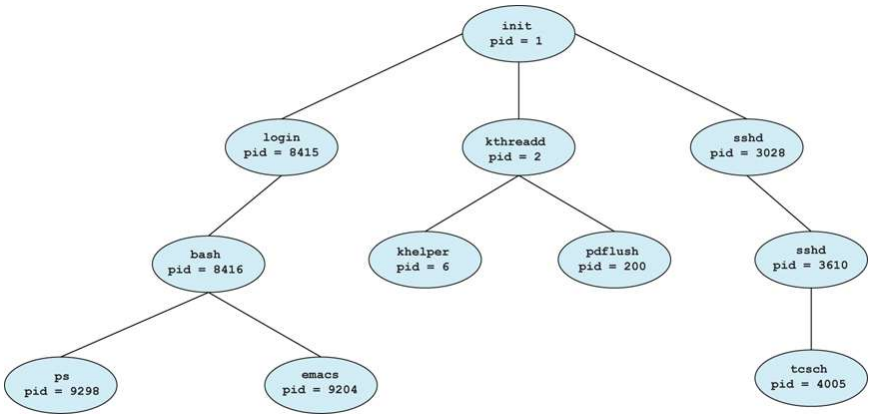
- A **process** can create another process.
- The process that creates is called the **parent**.
- The created process is called the **child**.
- Together, they form a **tree of processes**.

2. Process Identifier (PID)

- Each process has a unique **PID**.
- This helps the OS identify and manage processes.

3. Process Tree in UNIX

- UNIX systems maintain processes in a **tree structure**.
- A parent can create children, and children can create more children.



❖ Resource Sharing (Parent ↔ Child)

There are 3 options:

1. **Share all resources**: Parent and child use the same resources.
2. **Share a subset of resources**: Child only gets limited resources.
3. **Share no resources**: Parent and child work independently.

❖ Execution Options

1. **Parent and child run concurrently**: Both run in parallel.
2. **Parent waits until child finishes**: Parent uses **wait()** and pauses until the child completes.

❖ Address Space Options

A child process may have:

1. **Duplicate of parent's address space**
   - Exact copy of parent program + data.
   - This is what UNIX `fork()` does.
2. **New program loaded into address space**
   - Child replaces its memory with a new program.
   - UNIX: `exec()`
   - Windows: `CreateProcess()` / `spawn()`

❖ UNIX Example (fork + exec)

fork()

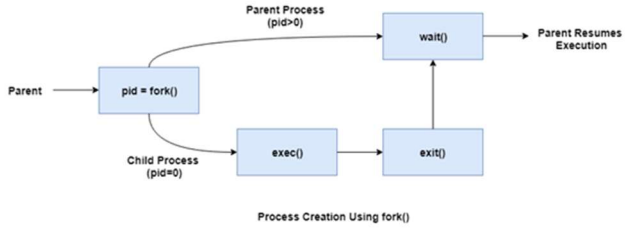
- Creates a new process.
- After `fork()`:
  - **Child's pid = 0**
  - **Parent gets child's pid (> 0)**
- Two processes now run the same code.

exec()

- Used **after fork()**.
- Replaces the child's memory with a new program (e.g., "ls" command).

wait()

- Parent waits until child finishes.



Process Creation Using fork()

❖ What Child Inherits in UNIX

- Privileges
- Scheduling attributes
- Open files
- Resources

❖ Summary of C Program (UNIX Process Creation)

1. Program calls `fork()`.
2. Child gets pid = 0; parent gets pid > 0.
3. Child uses `execlp("ls", ...)` to run the "ls" command.
4. Parent uses `wait()` to wait for child.
5. After child finishes, parent continues and exits.

❖ Creating a Process in Windows (CreateProcess API)

Steps in Windows:

1. Use **STARTUPINFO** and **PROCESS_INFORMATION** structures.
2. Call `CreateProcess()` with required parameters.
3. Parent waits for child using `WaitForSingleObject()`.
4. Close handles (`CloseHandle()`).

This C program launches Microsoft Paint (mspaint.exe) as a child process.

```c
int main(void) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;

    // Clear memory
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));

    // Create child process (MS Paint)
    if (!CreateProcess(
            NULL,      // Application name
            "C:\\WINDOWS\\system32\\mspaint.exe", // Command to run
            NULL,      // Process handle not inheritable
            NULL,      // Thread handle not inheritable
            FALSE,     // Set handle inheritance to FALSE
            0,         // No creation flags
            NULL,      // Use parent's environment
            NULL,      // Use parent's working directory
            &si,       // Pointer to STARTUPINFO structure
            &pi        // Pointer to PROCESS_INFORMATION structure
        ))
    {
        printf("CreateProcess failed.\n");
        return -1;
    }
```

```
    // Wait for the child process to finish
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child process complete.\n");

    // Close process and thread handles
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    return 0;
}
```

Process Termination

1. How a Process Terminates

- A process ends when it finishes its final instruction.
- It calls the **exit()** system call to tell the OS to delete it.
- The process can return a **status value** to the parent.
- All resources (memory, files, CPU registers) are **released** by the OS.

2. Parent Terminating a Child

A parent can forcefully terminate a child using **abort()**.
Reasons include:

1. Child exceeded its **resource limits**.
2. Child's task is no longer needed.
3. Parent is exiting, and the OS does not allow child to continue running.

3. Cascading Termination

- Some OS do not allow a child to exist after the parent dies.
- If the **parent terminates**, the OS automatically terminates:
  - All children
  - Grandchildren
  - Great-grandchildren
- This is called **cascading termination**.

4. Parent Waiting for Child

- A parent can wait for a child's termination using the **wait()** system call:

```
pid = wait(&status);
```

- This returns:
  - The **PID of the terminated child**
  - The **exit status** of the child

5. Zombie Process

- A child has **terminated**, but the **parent did not call wait()**.
- The child still has an entry in the process table → **Zombie**.
- Exists temporarily so parent can read the exit status.

6. Orphan Process

- The **parent terminates before calling wait()**.
- Child becomes an **orphan**.
- In UNIX, orphans are adopted by **init (PID 1)**.

Summary Table

Term	Meaning
exit()	Process ends normally
abort()	Parent kills the child
cascading termination	OS kills whole process family
wait()	Parent waits for child
zombie	Child ended, parent didn't wait
orphan	Parent ended, child continues

Inter-Process Communication (IPC)

⇨ Inter-Process Communication (IPC) is a mechanism that allows **processes to exchange data**, coordinate actions, and share resources.

❖ **It works:**

- Between processes on the **same system**, or
- Across **multiple computers** connected by a network.

❖ **IPC is needed for:**

- Sharing data
- Coordinating work
- Managing resources
- Synchronization
- Building modular systems
- Supporting distributed computing

Types of Processes

1. **Independent Processes** → Do not affect or interact with each other.
2. **Cooperating Processes** → Share data and can affect each other's execution.

Synchronization

⇨ Cooperating processes often share data or resources. To avoid **inconsistency**, they must synchronize.

❖ Why Synchronization Is Needed?

- To ensure **orderly execution** of cooperating processes.
- To maintain **data consistency**.
- To prevent issues like race conditions.

Producer–Consumer Problem (Illustration)

⇨ The Producer–Consumer problem is a classic example of why synchronization is necessary.

➢ Producer Process

- Produces data (items, messages, jobs).
- Places the data into a **buffer**.

➢ Consumer Process

- Consumes or uses the data.
- Removes data from the **buffer**.

➢ Need for Coordination

- Producer should not add data to a **full** buffer.
- Consumer should not remove data from an **empty** buffer.
  Thus, synchronization is required.

❖ Types of Buffers:

Unbounded Buffer	Bounded Buffer (Fixed Size)

• No practical limit on buffer size.
• Producer can generate data freely.
• Consumer takes data whenever available.
• Rare in practice due to memory limits.
	• Buffer has a **fixed capacity**.
	• Most common scenario.
	• Requires strict synchronization:
	  ○ Producer must wait if buffer is full.
	  ○ Consumer must wait if buffer is empty.

Bounded Buffer Problem

Shared Data

```c
#define BUFFER_SIZE 10

typedef struct {
    /* item structure */
} item;

item buffer[BUFFER_SIZE];
int in = 0;      // Points to next free position
int out = 0;     // Points to next full position
```

➔ **Limitation:** With this solution, only **9 out of 10** buffer slots can actually be used.
➔ **Reason:** We use `(in + 1) % BUFFER_SIZE == out` to detect **buffer full**, so one spot must remain empty to differentiate between full & empty.

Bounded Buffer – Producer Code

```c
item next_produced;

while (true) {
    /* produce an item in next_produced */

    while ( ((in + 1) % BUFFER_SIZE) == out )
        ;   /* do nothing - buffer is full */

    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Explanation

- Producer waits if buffer is **full**.
- Inserts item at `buffer[in]`.
- Moves `in` forward in a circular manner.

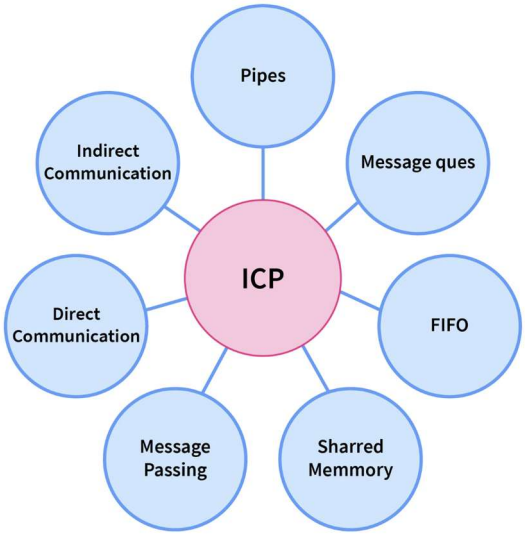
Bounded Buffer – Consumer Code

```c
item next_consumed;

while (true) {
    while (in == out)
        ;   /* do nothing - buffer is empty */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next_consumed */
}
```

Explanation

- Consumer waits if buffer is **empty**.
- Takes item from `buffer[out]`.
- Moves `out` forward in a circular manner.

Communications Models



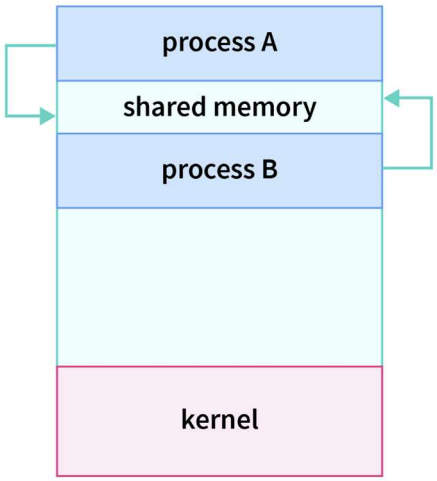
Shared Memory

⇨ Shared memory is a **region of memory accessible by multiple processes** at the same time. It allows fast communication because processes can **directly read and write** data in the shared area.

1. Very fast
2. Efficient
3. Requires careful synchronization (to avoid conflicts)

❖ How Shared Memory Works:

- Multiple processes attach the same memory block into their **address spaces**.
- One process writes data into the shared region.
- Other processes read the updated data from the same region.
- The **kernel is not used** for every read/write — only for setting up the shared memory.
- After creation, processes communicate *directly* → very fast.



❖ **Types of Shared Memory:**

A. Anonymous Shared Memory

- Not linked to any file.
- Created by the OS (e.g., using `shm_open`, `mmap`, `fork`).
- Only accessible to the processes that created or inherited it.
- Lives only while processes are running.

Example: Parent creates memory → child gets access after `fork()`.

B. Mapped Shared Memory

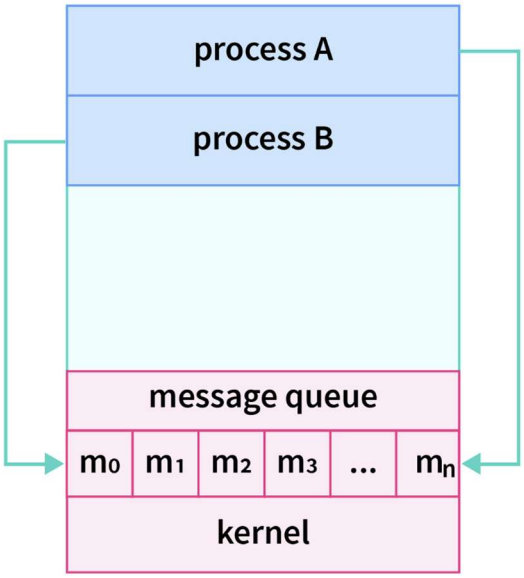
- Linked to a file or system object.
- Created by mapping a file into a process's address space.
- Accessible by multiple, unrelated processes.

Example: Two different programs map the same file into memory.

Message Passing

➔ It's an IPC method where processes **exchange messages** instead of sharing memory.

- **No shared variables** → works in distributed systems.
- Each process has a **unique process ID**.
- **Sending a message:** Specify recipient's process ID + message content → OS delivers it.
- **Receiving a message:** Process retrieves content and responds if needed.



❖ Advantages

1. Fast — messages copied directly between address spaces.
2. Flexible — any type of data can be shared.
3. Easy to implement — no special OS support required.

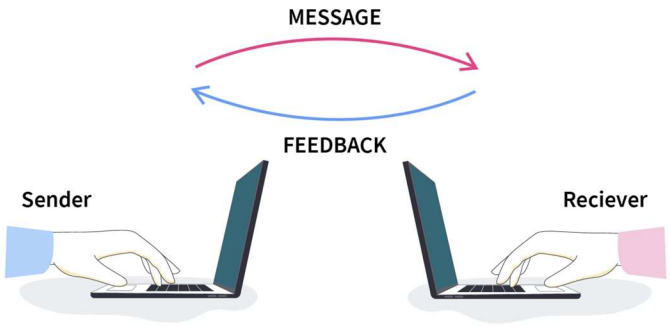
❖ Disadvantages

1. Hard to ensure correct **message order**.
2. Managing **message queue size** can be tricky.
3. Porting to different OS can be difficult.

Direct Communication

⇨ Processes communicate by **directly naming the sender or receiver**.

- **Link:** One communication link per pair of processes (usually bi-directional).
- **Requirement:** Sender must know **receiver's identifier** (process ID, port number, etc.).



❖ Message Syntax

- `send(P, message)` → send a message to process **P**
- `receive(Q, message)` → receive a message from process **Q**

❖ Properties of Communication Link

1. Links are **established automatically**.
2. Each link is associated with **exactly one pair of processes**.
3. **One link per pair** of communicating processes.
4. Links may be **unidirectional** or **bi-directional** (usually bi-directional).

Indirect Communication

⇨ Processes communicate **without naming each other directly** by using a **shared medium** like a **message queue** or **mailbox**.

❖ How It Works:

1. Sender places message in the **mailbox/port**.
2. Receiver retrieves (or deletes) the message from the mailbox.
3. Processes **don't need each other's identifiers**.

❖ Mailbox Operations

- **Create** a mailbox (port)
- **Send/Receive** messages through the mailbox
- **Delete** the mailbox

❖ Primitives

- `send(A, message)` → send a message to mailbox **A**
- `receive(A, message)` → receive a message from mailbox **A**

Indirect Communication Issues

➢ Assume a Scenario

- Processes **P1, P2, P3** share **mailbox A**
- P1 sends a message
- P2 and P3 can receive

➢ Problem

- Who actually receives the message?

➢ Solutions

1. **Restrict link** to at most **two processes**
2. Allow **only one process** to execute `receive` at a time
3. Let the **system select a receiver arbitrarily**
    - **Sender is notified** which process received the message

Blocking vs Non-blocking Message Passing

1. Blocking (Synchronous)

- **Blocking send:** Sender waits until message is received.
- **Blocking receive:** Receiver waits until a message is available.

2. Non-blocking (Asynchronous)

- **Non-blocking send:** Sender sends the message and continues immediately.
- **Non-blocking receive:** Receiver gets either:
    - A valid message, or
    - A null message if none is available

3. Combinations

- Any combination of blocking/non-blocking send/receive is possible.
- **Both send and receive blocking:** Called a **rendezvous** (synchronous exchange).

Buffering in Message Passing

⇨ Messages exchanged between processes are stored in a **temporary queue**.

❖ **Implemented in three ways:**

1. **Zero Capacity**
    - No messages are queued.
    - Sender waits for receiver (**rendezvous**).
2. **Bounded Capacity**
    - Queue has a finite length (n messages).
    - Sender waits if the link is full.
3. **Unbounded Capacity**
    - Queue can grow indefinitely.
    - Sender **never waits**.

Examples of IPC Systems

- **POSIX API** – Uses **shared memory** for IPC.
- **Mach OS** – Uses **message passing** for communication.
- **Windows IPC** – Uses **shared memory** to implement certain types of messages passing.
- **Pipes (UNIX)** – One of the earliest IPC mechanisms, used for **process communication**.

Mach OS

- **Communication:** Message-based, even **system calls are messages**.
- **Mailboxes:** Each task gets **two mailboxes** at creation:
    1. **Kernel**
    2. **Notify**

- **Message Transfer System Calls:**
    1. `msg_send()` – send a message
    2. `msg_receive()` – receive a message
    3. `msg_rpc()` – send a message and wait for reply

- **Mailbox Creation:** `port_allocate()`
- **Send/Receive Options if Mailbox Full:**
    1. Wait indefinitely
    2. Wait for at most n milliseconds
    3. Return immediately
    4. Temporarily cache the message

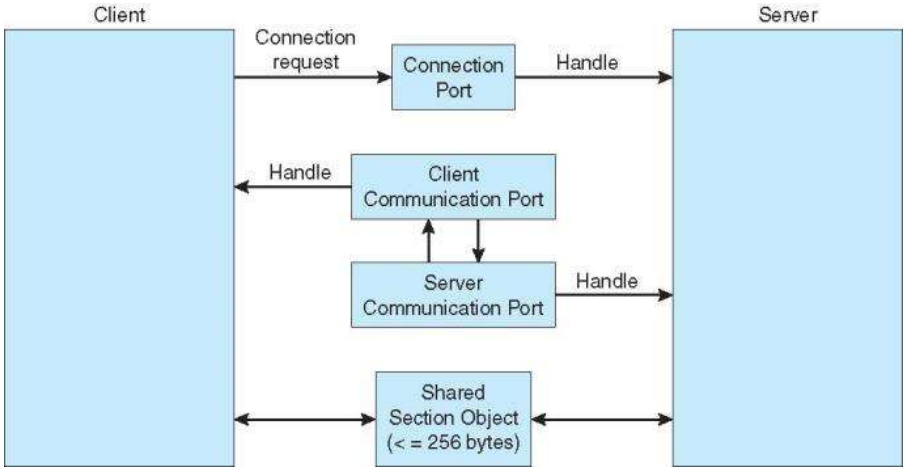
Windows IPC

- **Communication Style:** Mostly **message-passing**, using **Advanced Local Procedure Call (ALPC/LPC)**.
- **Scope:** Works **only between processes on the same machine**.
- **Ports:** Uses **ports (like mailboxes)** to create and manage communication channels.

❖ How Windows IPC Communication Works

1. **Client opens a handle** to the subsystem's **connection port**.
2. **Client sends a connection request** to the server.
3. **Server creates a private communication port** and sends the handle back to the client.
4. **Client and server communicate** using their port handles to:
    - Send messages
    - Receive replies
    - Perform callbacks

Local Procedure Calls in Windows



Communications in Client-Server Systems

1. Sockets

- Endpoints for communication.
- Used with TCP/UDP.
- Client connects to server socket and exchanges data.

2. Remote Procedure Calls (RPC)

- Call a function on another machine like a local call.
- Uses stubs to hide networking details.

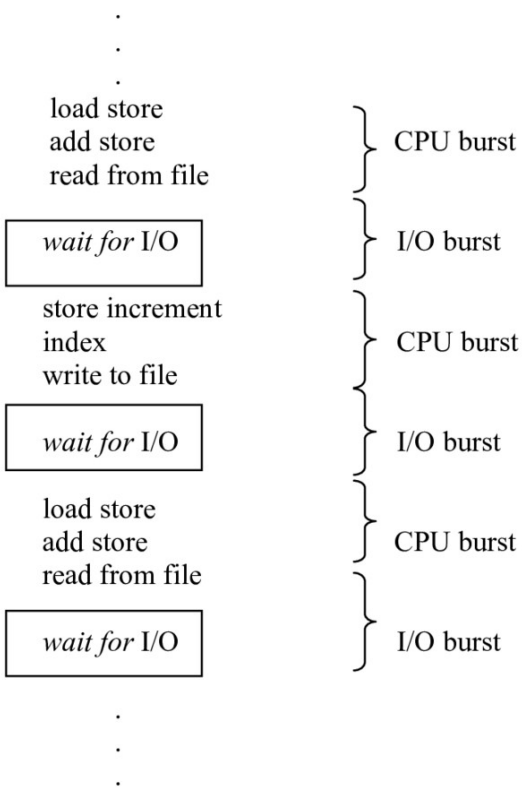
3. Remote Method Invocation (RMI – Java)

- Java version of RPC for objects.
- Allows calling methods on remote Java objects.

CPU SCHEDULING

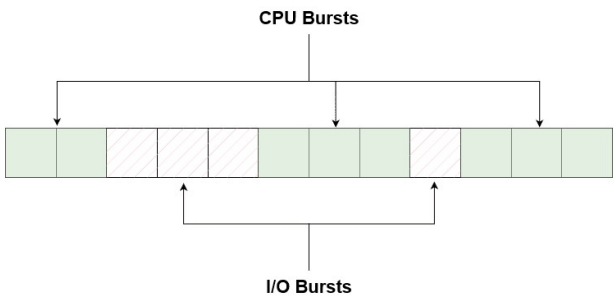
Basic Concepts

- **Goal of CPU Scheduling:** Keep the CPU busy all the time.
- **CPU–I/O Burst Cycle:**
  A process alternates between:
  - **CPU burst** → executing on CPU
  - **I/O burst** → waiting for I/O
- Most processes have **many short CPU bursts** and **few long ones**.



What is a CPU Burst?

⇨ A **CPU burst** is the amount of time a process spends **executing on the CPU** before doing an I/O operation.



Histogram of CPU-Burst Times

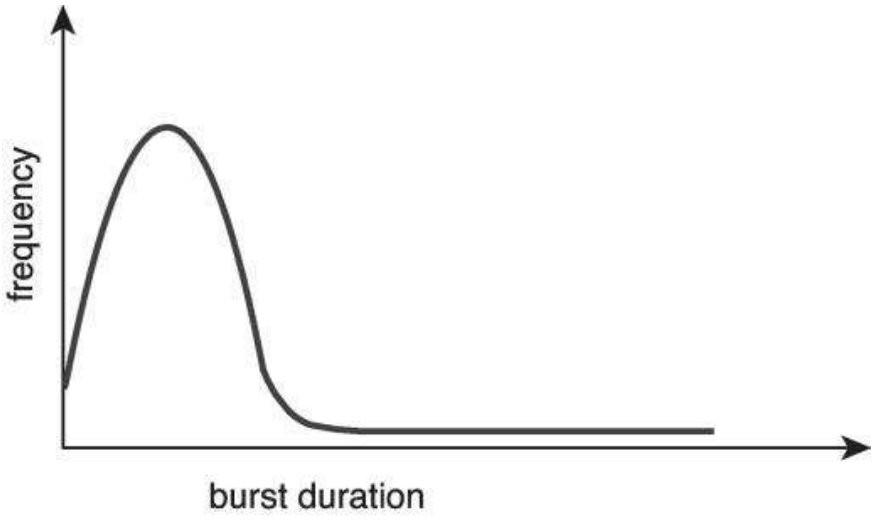
⇨ When we collect CPU burst times of many processes and plot them in a histogram, we usually see:

1. Large number of short CPU bursts

- Most processes only need the CPU for a **very short time**.
- Example: small calculations, quick system tasks.
- These bursts appear as a **high peak on the left side** of the histogram.

2. Small number of long CPU bursts

- A few processes need the CPU for a **long duration**.
- Example: heavy computations, big loops, scientific calculations.
- These appear as **bars on the right side**, but much fewer in number.



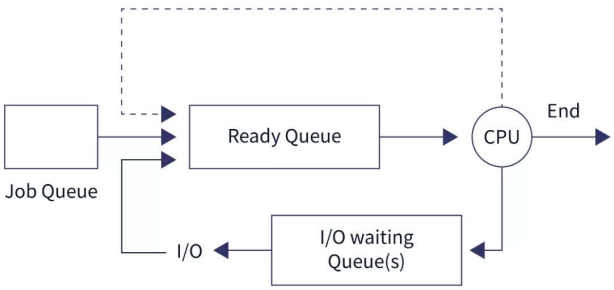
What is CPU Scheduling?

⇨ A **process** is a program in execution.

⇨ In multiprogramming systems, multiple processes stay in the **ready queue** waiting for CPU.

⇨ When the CPU finishes one job or a process waits for I/O, the CPU must pick the **next process**.

⇨ The method the OS uses to decide **which process runs next** is called **CPU Scheduling**.



Why CPU Scheduling is Needed

- To stop the CPU from becoming **idle** during I/O waits.
- To **maximize CPU utilization**.
- To ensure **fairness**, faster response, and better performance.

When CPU Scheduling Decisions Happen

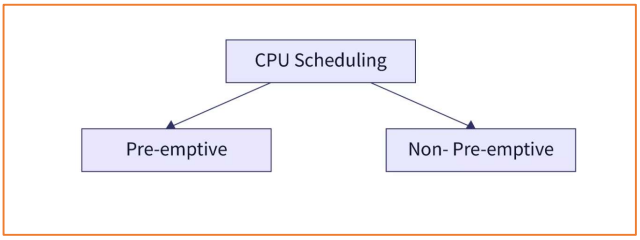
⇨ Scheduling occurs under 4 conditions:

1. **Running → Waiting State** (e.g., I/O request)
2. **Running → Ready State** (e.g., interrupt)
3. **Waiting → Ready State** (I/O completion)
4. **Running → Terminated**

❖ Conditions **1 & 4 → non-preemptive** (Must schedule next).
❖ Conditions **2 & 3 → preemptive**

Types of CPU Scheduling

1. Non-Preemptive Scheduling:



- A running process **cannot be interrupted**.
- CPU switches only when:
  - Process finishes OR
  - Process goes to waiting state.
- Examples:
  1. FCFS
  2. SJF (non-preemptive)
  3. Priority (non-preemptive)

Example idea: Each process runs completely before the next one starts.

2. Preemptive Scheduling

- The OS **can interrupt** a running process **before it finishes**.
- CPU can **take away** and given to another process.
- Useful when:
  - Higher-priority process arrives,
  - Time slice finishes (Round Robin),
  - Interrupt occurs.

Example idea: Timer interrupt forces process **A to stop** so process **B can run**.

CPU Scheduling Criteria

⇨ A good scheduling algorithm aims to:

1. **CPU Utilization** → Keep CPU as busy as possible.
2. **Throughput** → Number of processes completed per time unit.
3. **Turnaround Time** → Should be minimized.
4. **Waiting Time** → Should be minimized.
5. **Response Time** → Especially important for interactive systems; should be low.

Important Terminologies

- **Arrival Time (AT)** – Time a process enters the ready queue.
- **Burst Time (BT)** – CPU time needed to complete the process.
- **Completion Time (CT)** – Time when process finishes.
- **Turnaround Time (TAT)** = $CT - AT$
- **Waiting Time (WT)** = $TAT - BT = CT - AT - BT$
- **Response Time (RT)** – Time from arrival to first CPU allocation.

Types of CPU Scheduling Algorithms

1. **First Come, First Served (FCFS)**
2. **Shortest Job First (SJF)**
3. **Round Robin (RR)**
4. **Priority Scheduling**
5. **Priority Scheduling with Round Robin**
6. **Multilevel Queue (MLQ) Scheduling**
7. **Multilevel Feedback Queue (MLFQ) Scheduling**

First Come, First Served (FCFS)

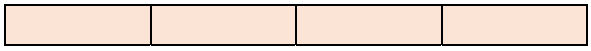
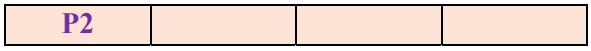
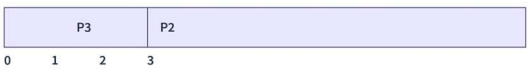
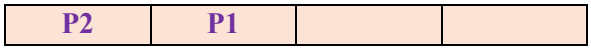
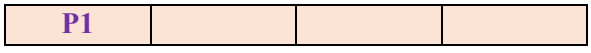
- FCFS (First Come First Serve) is a **non-preemptive** CPU scheduling algorithm.
- The process that **arrives first** is executed first.
- Once a process starts running, it **cannot be interrupted** until it finishes.

❖ **Real-life example:** Standing in line to buy a **movie ticket** → the person who comes first is served first

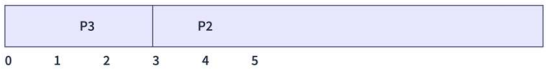
How FCFS Scheduling Works

⇨ We have 4 processes:

Process	Burst Time	Arrival Time
P1	6	2
P2	8	1
P3	3	0
P4	4	4

At time t = 0: Only **P3** has arrived so, it starts execution.	**At time t = 1:** P2 arrives but **P3 is still running**. So P2 waits in the queue.
	
Waiting Queue:	**Waiting Queue:**
	
At time t = 2: **P1** arrives. Still P3 is running, so P1 also waits.	**At time t = 3:** P3 completes. P2 is the next in queue → P2 starts running.
	
Waiting Queue:	**Waiting Queue:**
	

At time t = 4-5: **P4** arrives at time 4.
Still P2 is running, so P4 also waits.



Waiting Queue:

P1	P4		

At time t = 11: P2 completes.
P1 is the next in queue → P1 starts running.

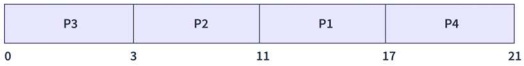
Waiting Queue:

P4			

At time t = 17: P1 completes.
P4 is the next in queue → P4 starts running.

Waiting Queue:

At time t = 21: P4 completes.



Waiting Queue:

Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	6	2	17	15
P2	8	1	11	10
P3	3	0	3	3
P4	4	4	21	17

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	6	2	17	15	9
P2	8	1	11	10	2
P3	3	0	3	3	0
P4	4	4	21	17	13

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{9 + 2 + 0 + 13}{4} = 6.0 \textbf{ ms}$$

$$\textbf{Average TAT} = \frac{15 + 10 + 3 + 17}{4} = 11.25 \textbf{ ms}$$

❖ Characteristics of FCFS

- Very **simple and easy** to implement.
- Processes are executed in **order of arrival**.
- Suitable for **batch systems**.

❖ Advantages

- Simple and easy scheduling method.
- Fair in terms of arrival order.
- No starvation (every process gets CPU eventually).

❖ Disadvantages

- Non-preemptive → small jobs wait behind long jobs.
- High waiting time for short processes (**convoy effect**).
- Not suitable for time-sharing systems.

Shortest Job First (SJF)

⇨ SJF chooses the **process with the smallest Burst Time** first.

There are two types:

1. **Non-Preemptive SJF** → Once a process starts, it runs until it finishes.
2. **Preemptive SJF (Shortest Remaining Time First – SRTF)** → CPU can be taken away if a shorter job arrives.

Non-Preemptive SJF Scheduling

❖ How it works:

- When the CPU becomes free, it picks the waiting process with the **shortest burst time**.
- Once a process starts, **it cannot be interrupted**.
- **Example:**

Process	Burst Time	Arrival Time
P1	2	2
P2	5	4
P3	3	0
P4	4	1

Step1: Time t = 0: Only **P3** has arrived so, it starts execution.

Waiting Queue:

Step2: Time t = 1: P4 arrives but **P3 is still running**. So P4 waits in the queue.

Waiting Queue:

P4			

Step3: Time t = 2: P1 arrives.
Still P3 is running, so P1 also waits.

Waiting Queue:

P4	**P1**		

Step4: Time t = 3: P3 completes.
Compare P1 (BT=2) and P4 (BT=4).
P1 is shorter → P1 starts.

Waiting Queue:

P4			

Step5: Time t = 4: P2 arrives
Still P1 is running, so P2 also waits.

Waiting Queue:

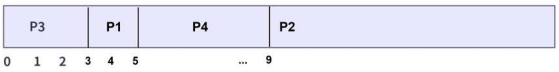
P4	**P2**		

Step6: Time t = 5: P1 completes.
Compare P2 (BT=5) and P4 (BT=4).
P4 is shorter → P4 starts.

Waiting Queue:

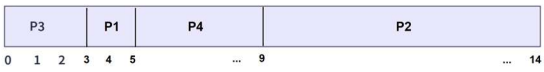
P2			

Step7: Time t = (6-9): P4 completes.
P2 is the next in queue → P2 starts running.



Waiting Queue:

Step8: Time t = 14: P2 completes.



Waiting Queue:

Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	2	2	5	3
P2	5	4	14	10
P3	3	0	3	3
P4	4	1	9	8

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	2	2	5	3	1
P2	5	4	14	10	5
P3	3	0	3	3	0
P4	4	1	9	8	4

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{1+5+0+4}{4} = 2.5\ \boldsymbol{units}$$

$$\textbf{Average TAT} = \frac{3+10+3+8}{4} = 6\ \boldsymbol{units}$$

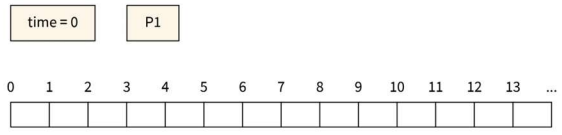
Preemptive SJF Scheduling

❖ How it works:

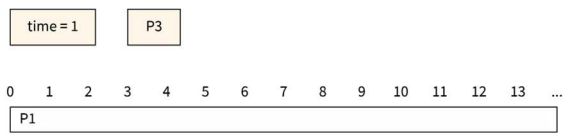
- The CPU always chooses the process with the **shortest remaining time**.
- If a new process arrives with a smaller burst time → **current process is preempted**.
- **Example:**

Process	Burst Time	Arrival Time
P1	3	0
P2	1	4
P3	5	1
P4	4	2

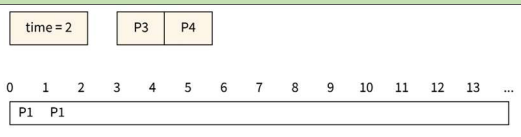
Step1: Time t = 0: Only **P1** has arrived so, it starts execution.



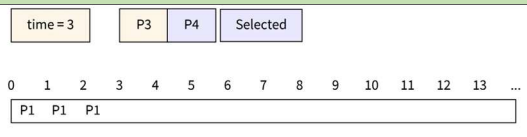
Step2: Time t = 1: P3 arrives (BT=5).
P1 (BT=2 remaining) is shorter → P1 continues.



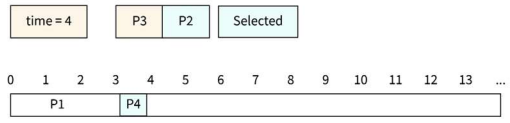
Step3: Time t = 2: P4 arrives (BT=4).
P1 (BT=1 remaining) is shorter → P1 continues.



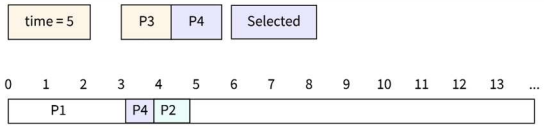
Step4: Time t = 3: P1 completes.
Compare P3 (BT=5) and P4 (BT=4).
P4 is shorter → P4 starts.



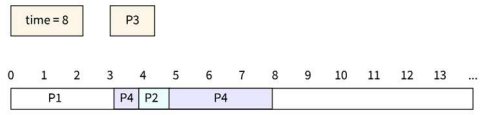
Step5: Time t = 4: P2 arrives (BT=1).
P4 (BT=3 remaining) > **P2(BT=1) → P2** Start.



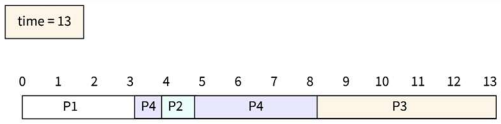
Step6: Time t = 5: P2 completes.
Compare P3 (BT=5) and P4 (BT=3).
P4 is shorter → P4 starts.



Step7: Time t = (6-8): P4 completes.
P3 is the next in queue → P3 starts running.



Step8: Time t = 13: P2 completes.



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	3	0	3	3
P2	1	4	5	1
P3	5	1	13	12
P4	4	2	8	6

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	3	0	3	3	0
P2	1	4	5	1	0
P3	5	1	13	12	7
P4	4	2	8	6	2

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{0+0+7+2}{4} = 2.25\ \boldsymbol{units}$$

$$\textbf{Average TAT} = \frac{3+1+12+6}{4} = \mathbf{5.5}\ \boldsymbol{units}$$

❖ Advantages of SJF

- Best **average waiting time** among all scheduling algorithms.
- Very efficient for **batch systems**.
- Minimizes turnaround time.

❖ Disadvantages of SJF

- Can cause **starvation** for long processes.
- Requires **accurate burst time prediction**, which is difficult.
- Not ideal for real-time or short-term scheduling.
- No process priority consideration.

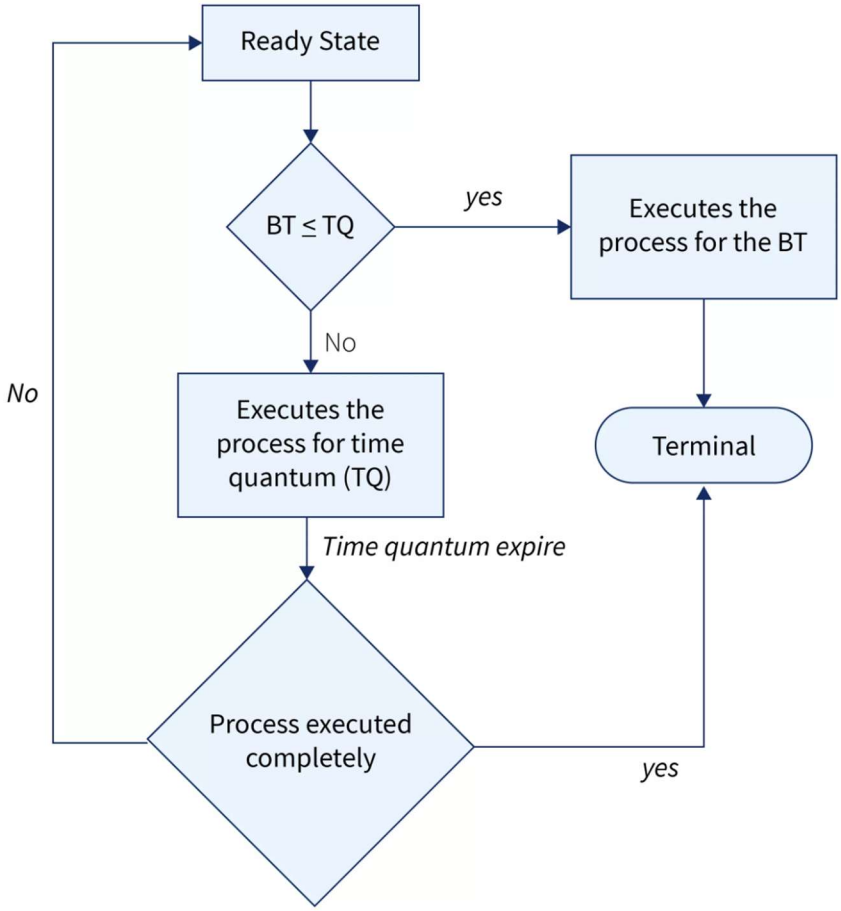
Round Robin (RR)

⇨ **Round Robin Scheduling** is a CPU scheduling algorithm where **each process gets a fixed time slice called Time Quantum (TQ)**. Processes are executed **cyclically**, in a **first-come, first-served (FCFS)** manner.

➢ If a process does not finish within its time quantum, it is **preempted**, and the CPU switches to the next process in the queue. The remaining part of the process is added back to the **ready queue**.

❖ How Round Robin Works

1. All processes enter the **ready queue**.
2. Each process gets the CPU for the **time quantum (TQ)**.
3. If **Burst Time ≤ Time Quantum**, the process finishes.
4. If **Burst Time > Time Quantum**, it runs for TQ, gets preempted, and goes to the back of the queue.
5. Continue until all processes are finished.



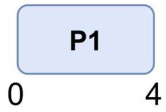
❖ **Example:** RR with Time Quantum = 4

Process	Arrival Time	Burst Time
P1	0	5
P2	1	6
P3	2	3
P4	3	1
P5	4	5
P6	6	4

➢ Solution:

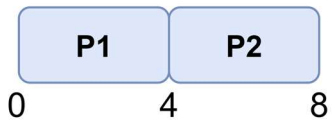
Step1: CPU runs **P1** for 4 units; Remaining BT **P1 = 5 − 4 = 1**

New arrivals:	P2 P3 P4 P5					
Queue:	**P2**	**P3**	**P4**	**P5**	**P1**	
Burst Time:	**6**	**3**	**1**	**5**	**1**	



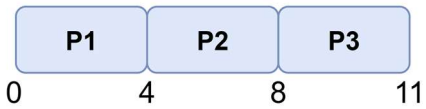
Step2: CPU runs **P2** for 4 units; Remaining BT **P2 = 6 − 4 = 2**

New arrivals:	P6					
Queue:	**P3**	**P4**	**P5**	**P1**	**P6**	**P2**
Burst Time:	**3**	**1**	**5**	**1**	**4**	**2**



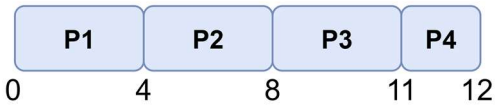
Step3: CPU runs **P3** for 4 units; **P3** needs only 3 → finished

New arrivals:						
Queue:	**P4**	**P5**	**P1**	**P6**	**P2**	
Burst Time:	**1**	**5**	**1**	**4**	**2**	



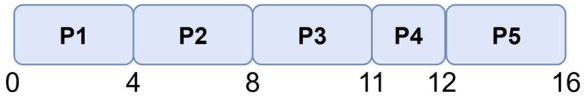
Step4: CPU runs **P4** for 4 units; **P4** needs only 1 → finished

New arrivals:						
Queue:	**P5**	**P1**	**P6**	**P2**		
Burst Time:	**5**	**1**	**4**	**2**		



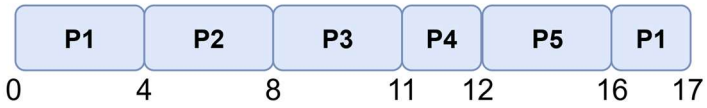
Step5: CPU runs **P5** for 4 units; Remaining BT $\mathbf{P5 = 5 - 4 = 1}$

New arrivals:						
Queue:	**P1**	**P6**	**P2**	**P5**		
Burst Time:	**1**	**4**	**2**	**1**		



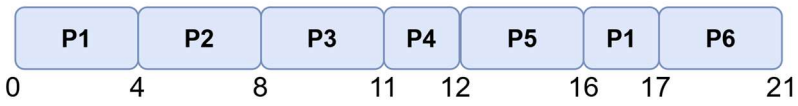
Step6: CPU runs **P1** for 4 units; **P1** needs only 1 → finished

New arrivals:						
Queue:	**P6**	**P2**	**P5**			
Burst Time:	**4**	**2**	**1**			



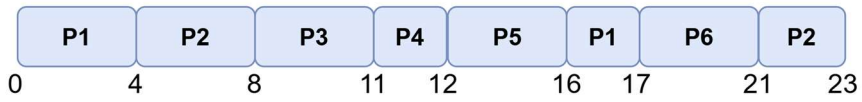
Step7: CPU runs **P6** for 4 units; **P6** needs only 4 → finished

New arrivals:						
Queue:	**P2**	**P5**				
Burst Time:	**2**	**1**				



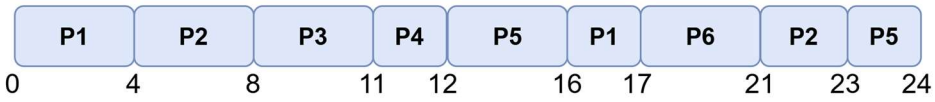
Step8: CPU runs **P2** for 4 units; **P2** needs only 2 → finished

New arrivals:						
Queue:	**P5**					
Burst Time:	**1**					



Step0: CPU runs **P5** for 4 units; **P5** needs only 1 → finished

New arrivals:						
Queue:						
Burst Time:						



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	17	17
P2	6	1	23	22
P3	3	2	11	9
P4	1	3	12	9
P5	5	4	24	20
P6	4	6	21	15

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	17	17	12
P2	6	1	23	22	16
P3	3	2	11	9	6
P4	1	3	12	9	8
P5	5	4	24	20	15
P6	4	6	21	15	11

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{12 + 16 + 6 + 8 + 15 + 11}{6} = \mathbf{11.33\ \textit{units}}$$

$$\textbf{Average TAT} = \frac{17 + 22 + 9 + 9 + 20 + 15}{6} = \mathbf{15.33\ \textit{units}}$$

❖ Advantages of Round Robin

- No starvation: every process eventually gets CPU time.
- Fair: equal priority for all processes.
- Supports preemption → good for time-sharing systems.
- Easy to implement.

❖ Disadvantages of Round Robin

- High waiting time if many processes are present.
- Too small-time quantum → too many **context switches.**
- No priority handling.

Priority Scheduling

⇨ The operating system must decide **which process gets the CPU next**. Priority Scheduling is a method where **each process is assigned a priority**, and the CPU is allocated to the **process with the highest priority**.

- ➢ Lower priority number = higher priority
- ➢ Higher priority number = lower priority

❖ **Example**: Priority 1 > Priority 2 > Priority 3

Types of Priority Scheduling

❖ Non-Preemptive Priority Scheduling

- Once a process with high priority starts **execution**, it will **not** be interrupted.
- New higher-priority processes must **wait** until the CPU becomes free.

❖ Preemptive Priority Scheduling

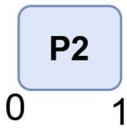
- If a process arrives with **higher priority**, it **preempts** (stops) the running process.
- The preempted process returns to the ready queue.

❖ Example: Non-Preemptive Priority Scheduling,

Process	Priority	Burst Time	Arrival Time
P1	3	10	0
P2	1	1	0
P3	4	2	0
P4	5	1	0
P5	2	5	0

➢ Solution:

Step1: CPU runs **P2,** since **Priority P2>P5>P1>P3>P4**



Step2: CPU runs **P5,** since **Priority P5>P1>P3>P4**

Step3: CPU runs **P1,** since **Priority P1>P3>P4**

Step4: CPU runs **P3,** since **Priority P3>P4**

Step5: CPU runs **P4,** since **Priority P4**

Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	10	0	16	16
P2	1	0	1	1
P3	2	0	18	18
P4	1	0	19	19
P5	5	0	6	6

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	10	0	16	16	6
P2	1	0	1	1	0
P3	2	0	18	18	16
P4	1	0	19	19	18
P5	5	0	6	6	1

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{6 + 0 + 16 + 18 + 1}{5} = \boldsymbol{8.2\ units}$$

$$\textbf{Average TAT} = \frac{16 + 1 + 18 + 19 + 6}{5} = \boldsymbol{12.0\ units}$$

❖ Advantages of Priority Scheduling

- Important tasks finish first
- Supports assigning importance to processes
- Useful when time and resource usage vary
- Works well in real-time systems

❖ Disadvantages of Priority Scheduling

- Risk of **starvation** (low priority processes may wait forever)
- If OS crashes, low-priority processes in RAM may be lost
- Complex priority handling may increase overhead

Priority Scheduling with Round Robin

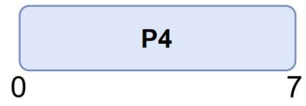
⇨ If two processes have **same priority**, they follow **Round Robin**

❖ Example: Preemptive Priority Scheduling, RR with Time Quantum = 2

Process	Priority	Burst Time	Arrival Time
P1	3	4	0
P2	2	5	0
P3	2	8	0
P4	1	7	0
P5	3	3	0

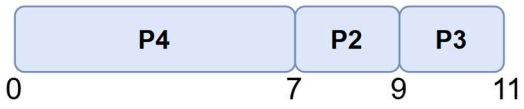
➢ Solution:

Step1: CPU runs **P4,** since **Priority P4>(P2=P3) > (P1= P5)**

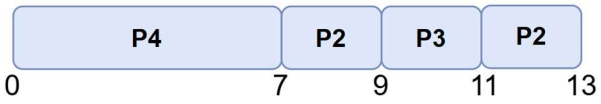


Step2: CPU runs **P2** for 2 units, since **Priority (P2=P3) > (P1= P5);** Remaining BT **P2 = 5 − 2 = 3**

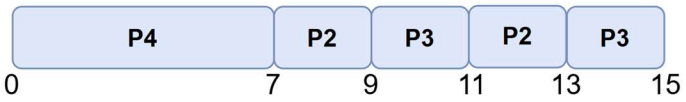
Step3: CPU runs **P3** for 2 units, since **Priority (P2=P3) > (P1= P5);** Remaining BT **P3 = 8 − 2 = 6**



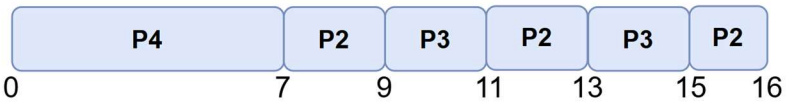
Step4: CPU runs **P2** for 2 units, since **Priority (P2=P3) > (P1= P5);** Remaining BT **P2 = 3 − 2 = 1**



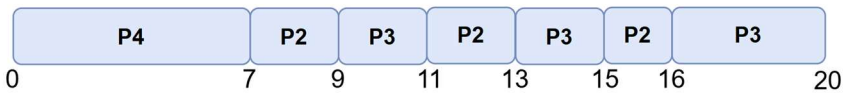
Step5: CPU runs **P3** for 2 units, since **Priority (P2=P3) > (P1= P5);** Remaining BT **P3 = 6 − 2 = 4**



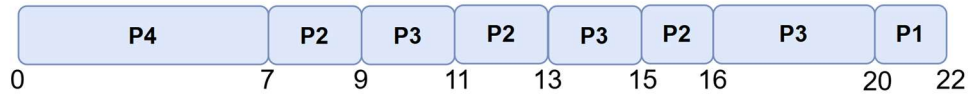
Step6: CPU runs **P2** for 2 units, since **Priority (P2=P3) > (P1= P5); P2** needs only 1 → finished



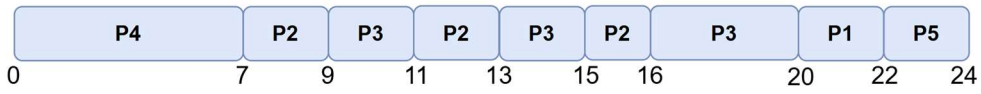
Step7: CPU runs **P3,** since **Priority P3> (P1= P5)**



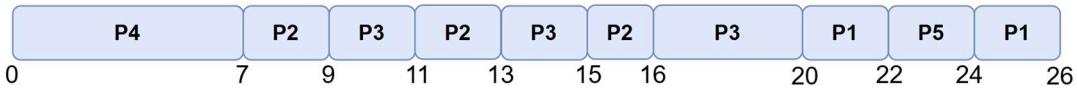
Step8: CPU runs **P1** for 2 units, since **Priority P1= P5;** Remaining BT **P1** = $\mathbf{4-2=2}$



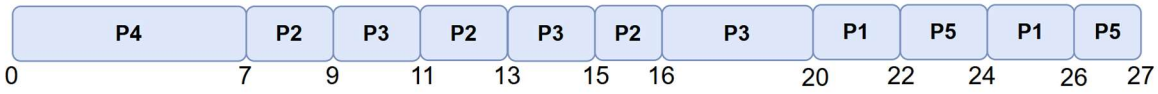
Step9: CPU runs **P5** for 2 units, since **Priority P1= P5;** Remaining BT **P5** = $\mathbf{3-2=1}$



Step10: CPU runs **P1** for 2 units, since **Priority P1= P5; P1** needs only 2 → finished



Step11: CPU runs **P5,** since **Priority o**nly P5 remaining



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	4	0	26	26
P2	5	0	16	16
P3	8	0	20	20
P4	7	0	7	7
P5	3	0	27	27

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	4	0	26	26	22
P2	5	0	16	16	11
P3	8	0	20	20	12
P4	7	0	7	7	0
P5	3	0	27	27	24

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{22+11+12+0+24}{5} = \mathbf{13.8}\ \boldsymbol{units}$$

$$\textbf{Average TAT} = \frac{26+16+20+7+27}{5} = \mathbf{19.2}\ \boldsymbol{units}$$

Problem1: Draw a Gantt chart illustrating the execution of these processes using **First-Come, First-Served (FCFS)** scheduling algorithm calculate the **average waiting time** and **average turnaround time**.

Process	Burst Time
P1	24
P2	3
P3	3

Solution:

Gantt chart:

Since AT is not provided, all processes are assumed to arrive at time 0.

Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	24	0	24	24
P2	3	0	27	27
P3	3	0	30	30

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	24	0	24	24	0
P2	3	0	27	27	24
P3	3	0	30	30	27

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{0 + 24 + 27}{3} = \textbf{17 } \boldsymbol{units}$$

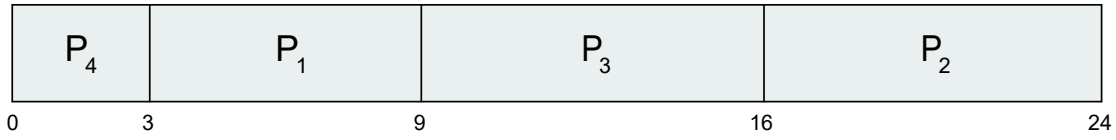
$$\textbf{Average TAT} = \frac{24 + 27 + 30}{3} = \textbf{27 } \boldsymbol{units}$$

Problem2: Draw a Gantt chart illustrating the execution of non-preemptive **Shortest-Job-First (SJF) scheduling** algorithm calculate the **average waiting time** and **average turnaround time**.

Process	Burst Time	Arrival Time
P1	6	0
P2	8	0
P3	7	0
P4	3	0

Solution:

Gantt chart:



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	6	0	9	9
P2	8	0	24	24
P3	7	0	16	16
P4	3	0	3	3

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	6	0	9	9	3
P2	8	0	24	24	16
P3	7	0	16	16	9
P4	3	0	3	3	0

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{3 + 16 + 9 + 0}{4} = \textbf{7 } \boldsymbol{units}$$

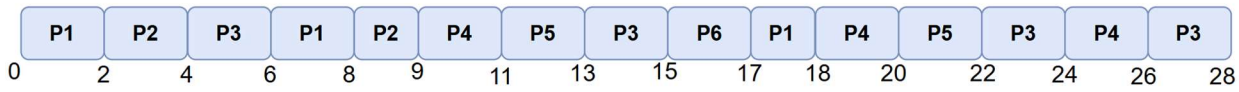
$$\textbf{Average TAT} = \frac{9 + 24 + 16 + 3}{4} = \textbf{13 } \boldsymbol{units}$$

Problem3: Draw a Gantt chart illustrating the execution of these processes using **Round-Robin scheduling** algorithm with quantum time **2 milliseconds**. Additionally, calculate the **average waiting time** and **average turnaround time**.

Process	Arrival Time	Burst Time (milliseconds)
P_1	0	5
P_2	1	3
P_3	2	8
P_4	5	6
P_5	6	4
P_6	8	2

Solution:

Gantt chart:



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	18	18
P2	3	1	9	8
P3	8	2	28	26
P4	6	5	26	21
P5	4	6	22	16
P6	2	8	17	9

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	18	18	13
P2	3	1	9	8	5
P3	8	2	28	26	18
P4	6	5	26	21	15
P5	4	6	22	16	12
P6	2	8	17	9	7

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{13 + 5 + 18 + 15 + 12 + 7}{6} = \mathbf{11.667\ ms}$$

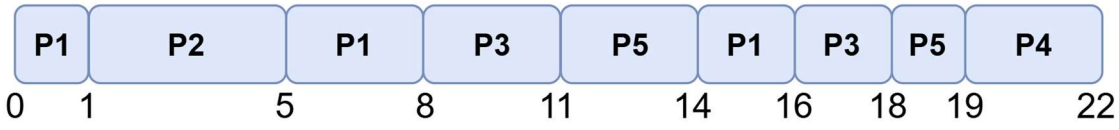
$$\textbf{Average TAT} = \frac{18 + 8 + 26 + 21 + 16 + 9}{6} = \mathbf{16.33\ ms}$$

Problem4: Draw a Gantt chart illustrating the execution of **Priority Scheduling and** if the priority is same use **Round Robin** algorithm with quantum time **3 milliseconds**. Additionally, calculate the **average waiting time** and **average turnaround time** for each process.

Process	Arrival Time	Burst Time	Priority
P_1	0 ms	6 ms	**2**
P_2	1 ms	4 ms	**1**
P_3	2 ms	5 ms	**2**
P_4	3 ms	3 ms	**3**
P_5	4 ms	4 ms	**2**

Solution:

Gantt chart: Since Priority Scheduling with Round Robin that means its Preemptive



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	6	0	16	16
P2	4	1	5	4
P3	5	2	18	16
P4	3	3	22	19
P5	4	4	19	15

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	6	0	16	16	10
P2	4	1	5	4	0
P3	5	2	18	16	11
P4	3	3	22	19	16
P5	4	4	19	15	11

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{10 + 0 + 11 + 16 + 11}{5} = \boldsymbol{9.6\ ms}$$

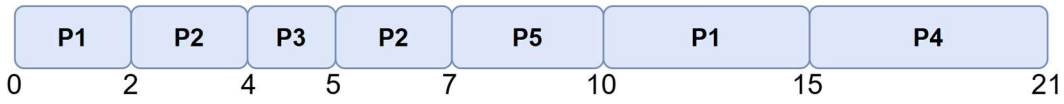
$$\textbf{Averag TAT} = \frac{16 + 4 + 16 + 19 + 15}{5} = \boldsymbol{14.0\ ms}$$

Problem5: Draw a Gantt chart illustrating the execution of **preemptive** Shortest-Job-First **(SJF) scheduling** algorithm calculate the **average waiting time** and **average turnaround time**.

Process	Arrival Time	Burst Time
P_1	0 ms	7 ms
P_2	2 ms	4 ms
P_3	4 ms	1 ms
P_4	5 ms	6 ms
P_5	6 ms	3 ms

➢ Solution:

Gantt chart:



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	7	0	15	15
P2	4	2	7	5
P3	1	4	5	1
P4	6	5	21	16
P5	3	6	10	4

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	7	0	15	15	8
P2	4	2	7	5	1
P3	1	4	5	1	0
P4	6	5	21	16	10
P5	3	6	10	4	1

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{8+1+0+10+1}{5} = \mathbf{4.0}\ \boldsymbol{ms}$$

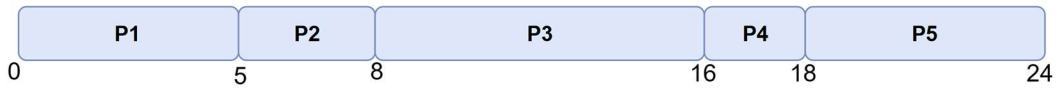
$$\textbf{Average TAT} = \frac{15+5+1+16+4}{5} = \mathbf{8.2}\ \boldsymbol{ms}$$

Problem6: Draw a Gantt chart illustrating the execution of these processes using **First-Come, First-Served (FCFS)** scheduling algorithm calculate the **average waiting time** and **average turnaround time**.

Process	Arrival Time	Burst Time
P_1	0 ms	5 ms
P_2	1 ms	3 ms
P_3	3 ms	8 ms
P_4	5 ms	2 ms
P_5	6 ms	6 ms

Solution:

Gantt chart:



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	5	5
P2	3	1	8	7
P3	8	3	16	13
P4	2	5	18	13
P5	6	6	24	18

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	5	5	0
P2	3	1	8	7	4
P3	8	3	16	13	5
P4	2	5	18	13	11
P5	6	6	24	18	12

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{0 + 4 + 5 + 11 + 12}{5} = \mathbf{6.4}\ \boldsymbol{ms}$$

$$\textbf{Average TAT} = \frac{5 + 7 + 13 + 13 + 18}{5} = \mathbf{11.2}\ \boldsymbol{ms}$$

Problem7: Answer the following questions using the given table below:

Process	Arrival Time	Burst Time (milliseconds)	Priority
P1	0	5	1
P2	2	4	2
P3	5	7	3
P4	7	8	1
P5	8	2	2
P6	10	4	1

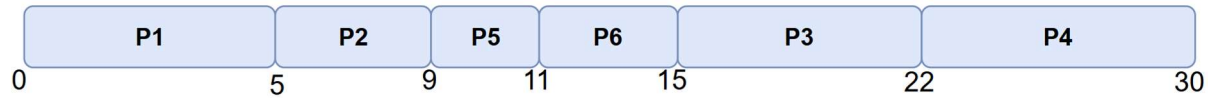
a) Draw four Gantt charts illustrating the execution of these processes using:
   a. preemptive SJF,
   b. non-preemptive Priority Scheduling (small priority number means high priority),
   c. Round Robin (quantum = 3) scheduling.

b) Calculate the waiting time and turn-around time of each process following the above scheduling algorithms.

c) Which of the algorithms gives the optimized result? Why?

Solution:

Gantt chart: Preemptive SJF



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	5	5
P2	4	2	9	7
P3	7	5	22	17
P4	8	7	30	23
P5	2	8	11	3
P6	4	10	15	5

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	5	5	0
P2	4	2	9	7	3
P3	7	5	22	17	10
P4	8	7	30	23	15
P5	2	8	11	3	1
P6	4	10	15	5	1

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{0 + 3 + 10 + 15 + 1 + 1}{6} = \mathbf{5.0}\ \boldsymbol{ms}$$

$$\textbf{Average TAT} = \frac{5 + 7 + 17 + 23 + 3 + 5}{6} = \mathbf{10.0}\ \boldsymbol{ms}$$

Gantt chart: Non-Preemptive Priority Scheduling

Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	5	5
P2	4	2	9	7
P3	7	5	30	25
P4	8	7	17	10
P5	2	8	23	15
P6	4	10	21	11

Waiting Time = TAT – BT

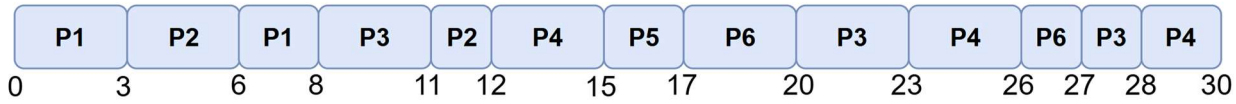
Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	5	5	0
P2	4	2	9	7	3
P3	7	5	30	25	18
P4	8	7	17	10	2
P5	2	8	23	15	13
P6	4	10	21	11	7

Average Waiting Time:

$$\textbf{Average WT} = \frac{0 + 3 + 18 + 2 + 13 + 7}{6} = \mathbf{7.167\ ms}$$

$$\textbf{Average TAT} = \frac{5 + 7 + 25 + 10 + 15 + 11}{6} = \mathbf{12.167\ ms}$$

Gantt chart: Round Robin



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	5	0	8	8
P2	4	2	12	10
P3	7	5	28	23
P4	8	7	30	23
P5	2	8	17	9
P6	4	10	27	17

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	5	0	8	8	3
P2	4	2	12	10	6
P3	7	5	28	23	16
P4	8	7	30	23	15
P5	2	8	17	9	7
P6	4	10	27	17	13

Average Waiting Time:

$$\textbf{Average WT} = \frac{3 + 6 + 16 + 15 + 7 + 13}{6} = \mathbf{10.00\ ms}$$

$$\textbf{Average TAT} = \frac{8 + 10 + 23 + 23 + 9 + 17}{6} = \mathbf{15.00\ ms}$$

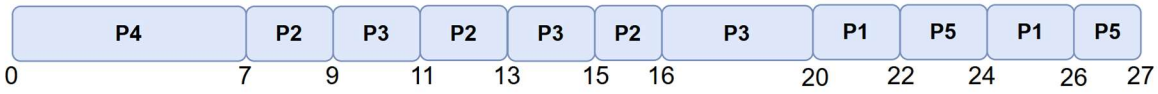
(c)

➔ **Preemptive SJF (SRTF)** gives the best (optimized) result here — it yields the lowest average waiting time (5.0) and lowest average turnaround time (10.0). Reason: SJF (and its preemptive variant SRTF) always runs the job with the smallest remaining time first, which tends to reduce the time other processes spend waiting and thus minimizes average waiting time (SJF is known to minimize average waiting time among non-priority algorithms when burst times are known).

Problem8: Draw a Gantt chart illustrating the execution of **Priority Scheduling with Round Robin** algorithm with **quantum time 2 milliseconds.** Additionally, calculate the average waiting time and average turnaround time.

Process	Priority	Burst Time	Arrival Time
P1	3	4	0
P2	2	5	0
P3	2	8	0
P4	1	7	0
P5	3	3	0

Gantt chart:



Turnaround Time = CT – AT

Process	BT	AT	CT	TAT = CT – AT
P1	4	0	26	26
P2	5	0	16	16
P3	8	0	20	20
P4	7	0	7	7
P5	3	0	27	27

Waiting Time = TAT – BT

Process	BT	AT	CT	TAT	WT = TAT – BT
P1	4	0	26	26	22
P2	5	0	16	16	11
P3	8	0	20	20	12
P4	7	0	7	7	0
P5	3	0	27	27	24

Average Waiting Time & Average Turnaround Time:

$$\textbf{Average WT} = \frac{22 + 11 + 12 + 0 + 24}{5} = \mathbf{13.8\ \textit{units}}$$

$$\textbf{Average TAT} = \frac{26 + 16 + 20 + 7 + 27}{5} = \mathbf{19.2\ \textit{units}}$$


SHELL

What is a Shell?

- ⇨ A **shell** is a program that acts as a **bridge between the user and the OS kernel**.
- ⇨ It starts when you **log in** or open a **terminal window**.
- ⇨ It allows you to run commands, programs, and scripts.

❖ Types of Shells

⇨ UNIX/Linux has two major shell families:

1. Bourne Shell Family **(default prompt: `$`)**

- `sh` – Bourne Shell
- `ksh` – Korn Shell
- `bash` – Bourne Again Shell (most popular)
- `sh` (POSIX Shell)

2. C Shell Family **(default prompt: `%`)**

- `csh` – C Shell
- `tcsh` – TENEX/TOPS C Shell

Kernel Version

⇨ Used to check which kernel your system is using.

Command:	**Output:**

`uname -r`	`mohsin@X1-Yoga:/$ uname -r`
	`6.6.87.2-microsoft-standard-WSL2`
	`mohsin@X1-Yoga:/$`

`Note:` Shows Linux **kernel version**.	

Check architecture (32-bit/64-bit):

⇨ Used to check which CPU architecture your system is using.

Command:	**Output:**

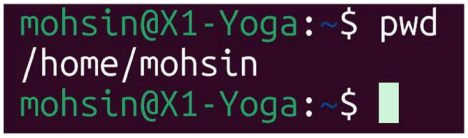
`uname -m`	`mohsin@X1-Yoga:/$ uname -m`
	`x86_64`
	`mohsin@X1-Yoga:/$`

`Note:` Shows CPU architecture (x86, x86_64, etc.)	

Current Directory

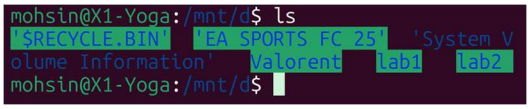
⇨ Your shell always works **inside a current directory**.

⇨ **To see your current directory**:

Command:	Output:
`pwd`	
Note: Example output: `/home/fred`	

The `ls` Command

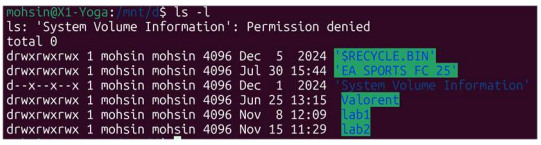
⇨ Used to see **list files and folders**.

Command:	Output:
`ls`	
Note: Shows **files** in the current directory.	

❖ Show hidden files:

Command:	Output:
`ls -a`	
Note: Shows all files **including** `.hidden` ones.	

❖ Show Detailed list:

Command:	Output:
`ls -l`	

Creating Directories

❖ Create a single directory:

Command:	Output:
`mkdir directoryName`	

❖ Create a multiple directory:

Command:	Output:
`mkdir dir1 dir2 dir3`	

Change Directory (cd)

⇨ Move from one directory to another.

Syntax:	Example:
`cd /folder/subfolder`	`cd /desktop/os`

❖ **To go back to home directory:**

Command:	Output:
`cd`	`mohsin@X1-Yoga:/mnt/d/Mohsin$ cd` `mohsin@X1-Yoga:~$`

❖ **To go to parent directory:**

Command:	Output:
`cd ..`	`mohsin@X1-Yoga:/mnt/d/Mohsin$ cd ..` `mohsin@X1-Yoga:/mnt/d$`

Create an Empty File

⇨ Use `touch` to make a zero-byte file.

Syntax:	Example:
`touch fileName.txt`	`touch test.txt`

⇨ **Q**: Create an empty zero-byte file named `os.txt`.

Command:	Output:

`touch os.txt`	`mohsin@X1-Yoga:/mnt/d/Mohsin$ touch os.txt`
	`mohsin@X1-Yoga:/mnt/d/Mohsin$ ls`
	`os.txt  test  test2  test3`

`cat` Command

⇨ The `cat` command is used to:

- Create a file
- View the content of a file
- Append content to a file
- Combine multiple files

❖ Create a File and Write Content into It:

Command:	Output:

`cat > test.txt`	`mohsin@X1-Yoga:/mnt/d/mih$ cat > test.txt`
	`hi,`
	`AIUB,`
	`CS, OS`
	`mohsin@X1-Yoga:/mnt/d/mih$`

Note: Type your text, then press **CTRL + D** to save	

❖ Viewing File Contents (cat):

Command:	Output:

`cat filename`	`mohsin@X1-Yoga:/mnt/d/mih$ cat test.txt`
	`hi,`
	`AIUB,`
	`CS, OS`
	`mohsin@X1-Yoga:/mnt/d/mih$`

❖ Another way (open file and type text):

Command:	Output:

`cat >> filename`	`mohsin@X1-Yoga:/mnt/d/mih$ cat >> test2.txt`
	`hello World`
	`StudyHub`
	`OS, Moshin`
	`mohsin@X1-Yoga:/mnt/d/mih$ cat test2.txt`
	`hello World`
	`StudyHub`
	`OS, Moshin`
	`mohsin@X1-Yoga:/mnt/d/mih$`

Note: Type your text, then press **CTRL + D** to save	

❖ Append Content to a File:

⇨ Add new text at the end of another file:

Command:	Output:
`cat file1.txt >> file2.txt`	

Note:

- **file1.txt** → *source file* (content will be copied from here)
- **file2.txt** → *destination file* (content will be appended here)

❖ Concatenate Multiple Files:

⇨ Join 2 files into a new file:

Command:	Output:
`cat file1 file2 > file3`	

Note:

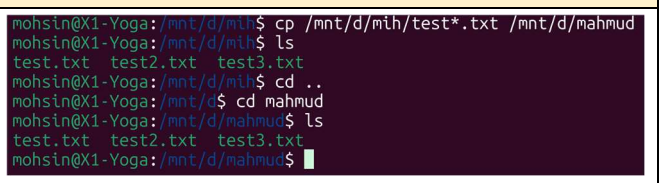
- **file1.txt → source file**
- **file2.txt → source file**
- **file3.txt → destination file (new file created with combined content)**

cp Command (Copy Files/Directories)

⇨ Used to copy **files** or **folders**.

Syntax:	Example:
`cp source destination`	`cp ex1 ex2`

⇨ **Q1**: Copy all the files `test.txt, test2.txt, & test3.txt` from `/mnt/d/mih` to `/mnt/d/mahmud.”`

Command:	Output:
`cp /mnt/d/mih/test*.txt /mnt/d/mahmud/`	

⇨ **Q2**: Copy all the files `test.txt, demo.txt, & os.txt` from `/mnt/d/mih` to `/mnt/d/mahmud.”`

Command:	Output:
`cp /mnt/d/mih/* /mnt/d/mahmud/`	

mv Command (Move/Rename Files)

⇨ Used to:

- Move a file
- Rename a file

Syntax:	Example:
`mv source destination`	`mv ex1 ex2`

⇨ **Q1**: Move all the files `test.txt, test2.txt, & test3.txt` from `/mnt/d/mih` to `/mnt/d/mahmud.”`

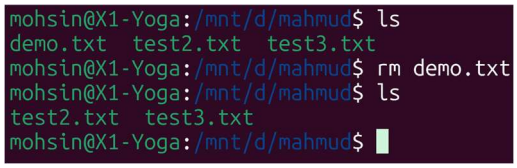
Command:	Output:
`mv /mnt/d/mih/* /mnt/d/mahmud/`	

⇨ **Q2**: Rename the file `test.txt` to `demo.txt` using the `mv` (move) command.

Command:	Output:
`mv test.txt demo.txt`	

`rm` Command (Remove Files/Directories)

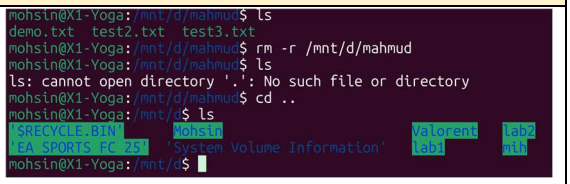
❖ Delete a file:

Command:	Output:
`rm filename`	

❖ Delete all files in a directory:

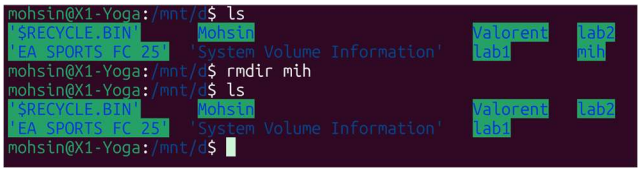
Command:	Output:
`rm *`	

❖ Delete directory and its contents:

Command:	Output:
`rm -r /mnt/d/mahmud`	

Note: **Be careful** this permanently deletes everything inside the folder also the directory

❖ **Remove directory:**

Command:	Output:
`rmdir dirname`	
Note: `rmdir` cannot delete directories that contain files.	

❖ **Remove directory with all subdirectories:**

Command:	Output:
`rmdir -p dir1/dir2/dir3`	

uname Command

⇨ Used to get basic system information.

❖ **Show machine (network) name:**

Command:	Output:
`uname -n`	

❖ **Show OS kernel version:**

Command:	Output:
`uname -r`	

System Information Commands

❖ Show current date:

Command:	Output:

`date`	`mohsin@X1-Yoga:/mnt/d/mih$ date`
	`Wed Nov 19 21:59:17 +06 2025`
	`mohsin@X1-Yoga:/mnt/d/mih$`

❖ Show current month calendar:

Command:	Output:

`cal`	`mohsin@X1-Yoga:/mnt/d/mih$ cal`
	`November 2025`
	`Su Mo Tu We Th Fr Sa`
	`1`
	`2 3 4 5 6 7 8`
	`9 10 11 12 13 14 15`
	`16 17 18 19 20 21 22`
	`23 24 25 26 27 28 29`
	`30`
	`mohsin@X1-Yoga:/mnt/d/mih$`

❖ Show system uptime:

Command:	Output:

`uptime`	`mohsin@X1-Yoga:/mnt/d/mih$ uptime`
	`22:01:54 up 2:55, 1 user, load average: 0.02, 0.01, 0.00`
	`mohsin@X1-Yoga:/mnt/d/mih$`

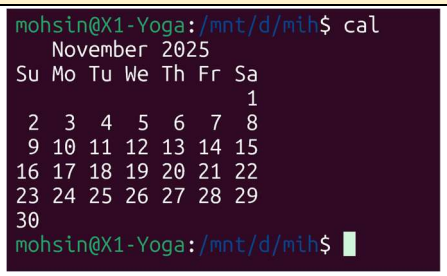
❖ Show logged-in username:

Command:	Output:

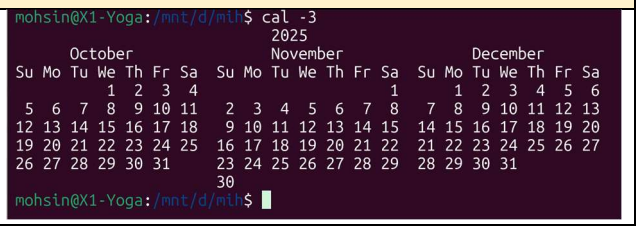
`whoami`	`mohsin@X1-Yoga:/mnt/d/mih$ whoami`
	`mohsin`
	`mohsin@X1-Yoga:/mnt/d/mih$`

Calendar (cal) Commands

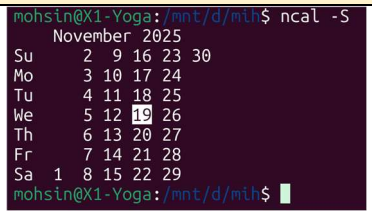
❖ Basic calendar:

Command:	Output:
`cal`	

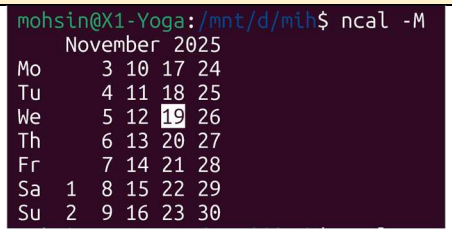
❖ Show previous, current & next month:

Command:	Output:
`cal -3`	

❖ Calendar starting with Sunday:

Command:	Output:
`ncal -S`	

❖ Calendar starting with Monday:

Command:	Output:
`ncal -M`	

Date (date) Commands

❖ **Show system date:**

Command:	Output:
`date`	

❖ **Show month (number):**

Command:	Output:
`date +%m`	

❖ **Show month name + month number:**

Command:	Output:
`date +%h%m`	

❖ **Show month name:**

Command:	Output:
`date +%h`	

❖ **Show hour:**

Command:	Output:
`date +%H`	

❖ **Show minutes:**

Command:	Output:
`date +%M`	

❖ **Show time in AM/PM format:**

Command:	Output:
`date +%r`	

❖ **Show day of the month:**

Command:	Output:
`date +%d`	

who Commands

❖ **Show login details:**

Command:	Output:

`who`	`mohsin@X1-Yoga:/mnt/d/mih$ who`
	`mohsin pts/1 2025-11-19 19:05`
	`mohsin@X1-Yoga:/mnt/d/mih$`

❖ **Show current user (who am i):**

Command:	Output:

`whoami`	`mohsin@X1-Yoga:/mnt/d/mih$ whoami`
	`mohsin`
	`mohsin@X1-Yoga:/mnt/d/mih$`

❖ **Show my login ID:**

Command:	Output:

`logname`	`mohsin@X1-Yoga:/mnt/d/mih$ logname`
	`logname: no login name`
	`mohsin@X1-Yoga:/mnt/d/mih$`

history Command

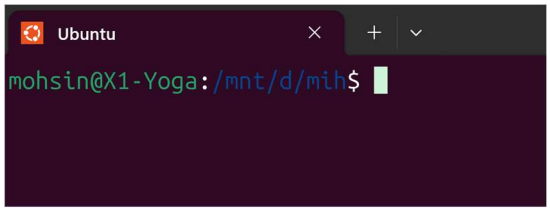
⇨ Shows all commands previously used in the current terminal session.

Command:	Output:

`history`	`378 clear`
	`379 cal -s`
	`380 cal -S`
	`381 ncal -S`
	`382 clear`
	`383 ncal -M`
	`384 ncal -S`
	`385 date +%m`
	`386 date +%h%m`
	`387 date +%h`
	`388 date +%H`
	`389 clear`
	`390 date +%M`
	`391 date +%r`
	`392 date +%d`
	`393 who`

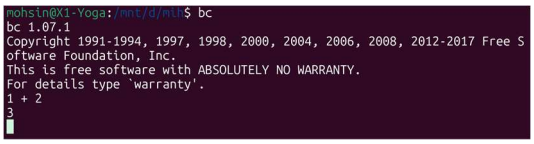
clear Command

⇨ Clears the terminal screen.

Command:	Output:
`clear`	

bc Command

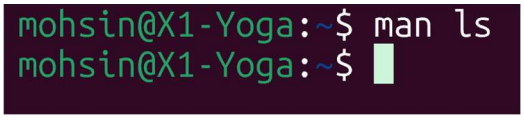
⇨ Used as a basic calculator.

Command:	Output:
`bc` `1 + 2`	

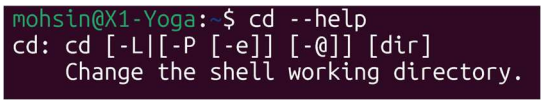
man and --help Command

⇨ Used to get help about any command.

❖ **Show manual (full documentation):**

Command:	Output:
`man command_name`	

❖ **Quick help (short usage info):**

Command:	Output:
`command_name --help`	

wc – Count Words, Lines, and Bytes

❖ **Show number of lines, words, and bytes:**

Command:	Output:

`wc filename`	`mohsin@X1-Yoga:/mnt/d/os$ cat demo.txt`
	`alif jam`
	`nihan`
	`aiub`
	`mohsin@X1-Yoga:/mnt/d/os$ wc demo.txt`
	`3  4 20 demo.txt`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ **Count characters:**

Command:	Output:

`wc -c filename`	`mohsin@X1-Yoga:/mnt/d/os$ cat test.txt`
	`MOHSIN`
	`mohsin@X1-Yoga:/mnt/d/os$ wc -c test.txt`
	`7 test.txt`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ **Count lines:**

Command:	Output:

`wc -l filename`	`mohsin@X1-Yoga:/mnt/d/os$ cat demo.txt`
	`alif jam`
	`nihan`
	`aiub`
	`mohsin@X1-Yoga:/mnt/d/os$ wc -l demo.txt`
	`3 demo.txt`
	`mohsin@X1-Yoga:/mnt/d/os$`

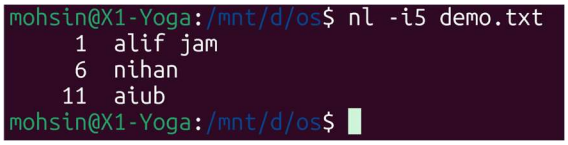
nl – Display Line Numbers

❖ **Show lines with numbers:**

Command:	Output:

`nl filename`	`mohsin@X1-Yoga:/mnt/d/os$ nl demo.txt`
	`1  alif jam`
	`2  nihan`
	`3  aiub`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ **Increase numbering by 5:**

Command:	Output:
`nl -i5 filename`	

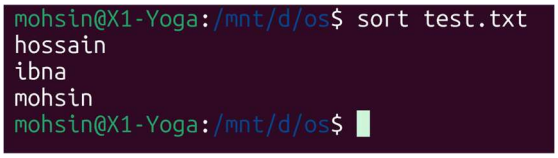
Sort Command

⇨ Used to sort text alphabetically.

❖ **Sort in reverse order:**

Command:	Output:
`sort -r filename`	

❖ **Sort normally:**

Command:	Output:
`sort filename`	

❖ **Sort and remove duplicates:**

Command:	Output:
`sort -u filename`	

❖ **Sort by 2nd field (full word) alphabetically:**

Command:	Output:

`sort -k 2 filename`	`mohsin@X1-Yoga:/mnt/d/os$ cat > data.txt`
	`101 apple red`
	`102 banana yellow`
	`103 apricot orange`
	`104 berry purple`
	`105 avocado green`
	`mohsin@X1-Yoga:/mnt/d/os$ sort -k 2 data.txt`
	`101 apple red`
	`103 apricot orange`
	`105 avocado green`
	`102 banana yellow`
	`104 berry purple`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ **Sort by 2nd field starting from the 3rd character:**

Command:	Output:

`sort -k 2.3 filename`	`mohsin@X1-Yoga:/mnt/d/os$ sort -k 2.3 data.txt`
	`102 banana yellow`
	`104 berry purple`
	`101 apple red`
	`103 apricot orange`
	`105 avocado green`

head Command

❖ **Show first 10 lines:**

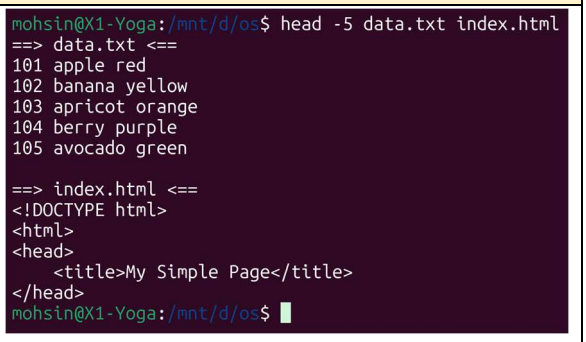
Command:	Output:

`head filename`	`mohsin@X1-Yoga:/mnt/d/os$ head index.html`
	`<!DOCTYPE html>`
	`<html>`
	`<head>`
	`<title>My Simple Page</title>`
	`</head>`
	`<body>`
	`<h1>Hello, World! ☀️</h1>`
	`<p>This is a simple HTML page.</p>`
	
	`<h2>About This Page</h2>`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ **Show first 6 characters:**

Command:	Output:
`head -6c filename`	`mohsin@X1-Yoga:/mnt/d/os$ head -6c index.html` `<!DOCTmohsin@X1-Yoga:/mnt/d/os$`

❖ **Show 5 lines from two files:**

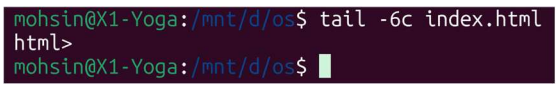
Command:	Output:
`head -5 file1 file2`	

`tail` Command

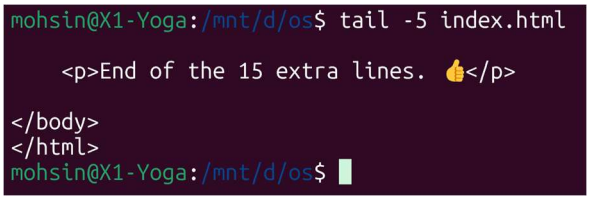
❖ **Show last 10 lines:**

Command:	Output:
`tail filename`	

❖ **Show last 6 characters:**

Command:	Output:
`tial -6c filename`	

❖ **Show last 5 lines from a files:**

Command:	Output:
`tail -5 filename`	

cut Command

⇨ Used to extract specific columns or fields.

❖ Extract fields using a delimiter:

Command:	Output:
`cut -d, -f1,3 filename`	

Note:

- `-d,` → delimiter is a **comma**
- `-f1,3` → extract **field 1** and **field 3**

❖ Extract characters by position:

Command:	Output:
`cut -c 1-4 cutfile.txt`	

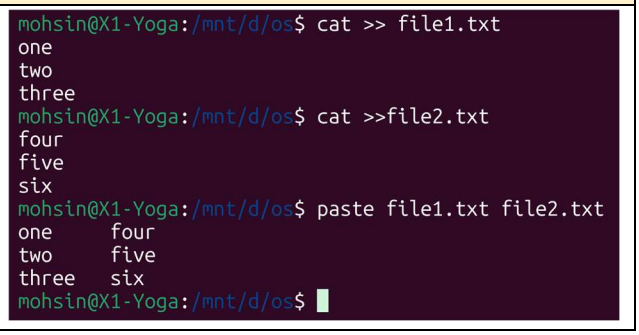
Note:

- `-c` → extract by **character position**
- `1-4` → show characters from position 1 to 4
- No delimiter needed

`paste` Command

⇨ Used to join **files horizontally** (line by line).

❖ Normal paste (default delimiter = TAB):

Command:	Output:
`paste file1 file2`	

❖ Using a custom delimiter (comma):

Command:	Output:
`paste -d, file1 file2`	

Breakdown:

- `-d,` → use **comma (,)** instead of tab
- `paste` joins the **first line** of file1 with the **first line** of file2
- Then second line with second line, and so on

❖ Using a custom delimiter (colon):

Command:	Output:
`paste -d“:” file1 file2`	

grep Command

⇨ `grep` is used to **search for a specific word or pattern inside a file**.

It will **show only the lines** where the pattern is found.

❖ **Basic syntax:**

Syntax:	Example:
`grep "pattern" filename`	`grep "AIUB" demo.txt`
• **pattern** → the text you want to search for • **filename** → the file where you want to search	

➢ Suppose we have a file named **students.txt** with the following content:

```
Alif passed the exam
Mohsin is preparing for the exam
Jam failed the exam
Mohsin wants to improve
Nihan passed the exam
```

Search for "Mohsin":

Command:	Output:

`grep "Mohsin" students.txt`	`mohsin@X1-Yoga:/mnt/d/os$ grep "Mohsin" students.txt`
	`Mohsin is preparing for the exam`
	`Mohsin wants to improve`
	`mohsin@X1-Yoga:/mnt/d/os$`

Search for "passed":

Command:	Output:

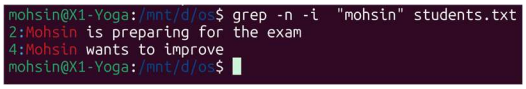
`grep "passed" students.txt`	`mohsin@X1-Yoga:/mnt/d/os$ grep "Passed" students.txt`
	`mohsin@X1-Yoga:/mnt/d/os$ grep "passed" students.txt`
	`Alif passed the exam`
	`Nihan passed the exam`
	`mohsin@X1-Yoga:/mnt/d/os$`

❖ `grep -i` → Case-insensitive search:

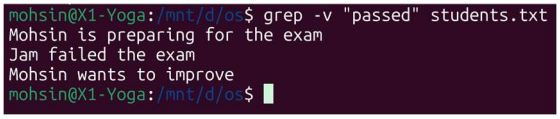
Command:	Output:
`grep -i "Passed" students.txt`	

❖ `grep -n` → Show line numbers:

⇨ Shows the line number where the pattern is found.

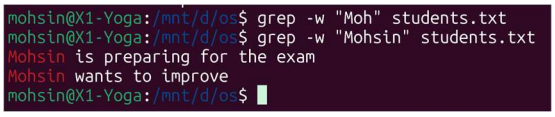
Command:	Output:
`grep -n "Passed" students.txt`	

❖ `grep -v` → Show lines that do NOT contain the pattern (Invert the Match):

Command:	Output:
`grep -v "passed" students.txt`	

❖ `grep -w` → Match whole word only:

⇨ Matches the exact word, not part of another word.

Command:	Output:
`grep -w "Mohsin" students.txt`	

❖ `grep -c` → Count how many lines contain the pattern:

Command:	Output:
`grep -c "Mohsin" students.txt`	

❖ `grep -o` → Display Only the Matched Text:

⇨ Shows **only the matching word**, not the whole line.

Command:	Output:
`grep -o "Passed" students.txt`	

File Permissions in Linux

⇨ Linux supports many users accessing the system at the same time, so it uses:

➢ Two levels of security:

1. **Ownership:** Every file and directory must have an owner that determines who controls the file.
2. **Permissions:** Permissions define what actions the owner (and others) can perform on the file.

Ownership Types

➔ Every file/directory belongs to **three types of owners**:

1. User (u)

- The person who created the file
- Owner of the file

2. Group (g)

- A group containing multiple users
- All group members share the same permissions

3. Others (o)

- Anyone else who can access the file
- Not the owner & not in the group

Permission Types (r, w, x)

Permission	Symbol	Meaning	Value
Read	r	View file contents	4
Write	w	Modify file	2
Execute	x	Run file / enter directory	1

Numeric Permissions

⇨ Numeric permissions come from adding values of **r, w, x**.

Example:

```
rwx = 4 + 2 + 1 = 7
rw- = 4 + 2 + 0 = 6
r-- = 4 + 0 + 0 = 4
```

Changing Permissions — chmod Command

⇨ `chmod` is used to modify file permissions.

Syntax:

```
chmod <permissions> <filename>
```

Check Current Permission

Command:

```
ls -l
```

Output:

```
mohsin@X1-Yoga:/mnt/d/os$ ls -l
total 0
-rwxrwxrwx 1 mohsin mohsin  87 Nov 20 21:09 cutfile.txt
-rwxrwxrwx 1 mohsin mohsin  86 Nov 20 20:30 data.txt
```

Part	Meaning
`-`	Regular file (not a directory)
`rwx`	User (owner) permissions
`rwx`	Group permissions
`rwx`	Others permissions

So for cutfile.txt:

- **User → rwx → 777 → read, write, execute**
- **Group → rwx → 777 → read, write, execute**
- **Others → rwx → 777 → read, write, execute**

This means everyone can read, write, and execute the file.

Why this is risky?

⇨ `777` means **anyone can delete or edit your files**, especially on shared systems.

For safety, normal data files should be:	Or directories:
`644 (rw-r--r--)`	`755 (rwxr-xr-x)`

❖ Apply using chmod:

Command:	Output:

`chmod 644 cutfile.txt`	`mohsin@X1-Yoga:~/os$ ls -l`
	`total 0`
	`-rwxrwxrwx 1 mohsin mohsin 0 Nov 21 00:11 cutfile.txt`
	`-rwxrwxrwx 1 mohsin mohsin 0 Nov 21 00:11 demo.txt`
	`mohsin@X1-Yoga:~/os$ chmod 644 cutfile.txt`
	`mohsin@X1-Yoga:~/os$ ls -l`
	`total 0`
	`-rw-r--r-- 1 mohsin mohsin 0 Nov 21 00:11 cutfile.txt`
	`-rwxrwxrwx 1 mohsin mohsin 0 Nov 21 00:11 demo.txt`
	`mohsin@X1-Yoga:~/os$`

Exercise

Example 1: Find the numeric value for: **rw- rw- ---**

⇨ **Solution**:

- User → `rw-` = 4 + 2 + 0 = **6**
- Group → `rw-` = 4 + 2 + 0 = **6**
- Others → `---` = 0 + 0 + 0 = **0**

∴ Final value = **660**

Example 2: Find the numeric value for: rw- rw- r--

⇨ **Solution:**

- User → `rw-` = 4 + 2 + 0 = **6**
- Group → `rw-` = 4 + 2 + 0 = **6**
- Others → `r--` = 0 + 0 + 0 = **4**

∴ Final value = **664**

Example 3: Create a file os1.txt inside dir2sub

Give permissions:

- User → read + write + execute
- Group → read + execute
- Others → read only

⇨ **Solution:**

- User → read + write + execute → **rwx → 7**
- Group → read + execute → **r_x → 5**
- Others → read only → **r → 4**

Command:	
File creating inside dir2sub:	Change permission:
`touch dir2sub/os1.txt`	`chmod 754 dir2sub/os1.txt`

Problem1: Write a shell command to execute a case-insensitive search for the string **"aiub"** in a file (for example, *information.txt*).

Answer:

```
grep -i “aiub” information.txt
```

Problem2: How can you display the first five lines of multiple files (for example *file1.txt, file2.txt, file3.txt*)?

Answer:

```
head -5 file1.txt file2.txt file3.txt
```

Problem3: Write a shell command to extract characters **5 through 10** from each line of a text file (for example – *data.txt*)

Answer:

```
cut -c 5-10 data.txt
```

Problem4: Write a shell command that will sort the contents of osfile.txt and remove duplicate lines.

Answer:

```
sort -u osfile.txt
```

Problem5: Write a shell command to merge two files named file1.txt and file2.txt.

Answer:

```
cat file1.txt file2.txt > marged.txt
```

Problem6: Write a shell command to display all lines from output.txt that match the pattern string “Darwin”.

Answer:

```
grep “Darwin” output.txt
```

Problem7: Create a directory named backup and give read, write and execute access only to the user.

Answer:

```
mkdir backup
chmod 700 backup
```

Problem8: Create a directory named project and give read, write and execute access only to the group.

Answer:

```
mkdir project
chmod 70 project
```

Problem9: Write a shell command to display the first 10 lines of a file named **system.log**.

Answer:

```
head system.log
```

Problem10: Write a shell command to count the total number of lines in a file named **notes.txt**.

Answer:

```
wc -l notes.txt
```